
How the Engine Works

Digest edition

The internal mechanisms of the Lynx Harness agent engine — condensed from the 294-page edition with every citation, control-decision table, diagram and finding intact.

Every request this engine sends to a model is recomputed from an append-only event log immediately before dispatch — and that single decision is what makes resume, fork, compaction, and replay variations of one operation.

Contents

How to read this

What is kept, and what was cut

Reading paths

Evidence

Part I • Orientation

- 1 The core idea
- 2 Just enough architecture
- 3 Core data structures

Part II • The mental model

- 4 The engine in ~100 lines

Part III • The mechanisms

- 5 The append-only log
- 6 The surface
- 7 From log to request
- 8 Projections
- 9 The turn and step loops
- 10 Building the request
- 11 The reconstruction invariant
- 12 The inbox
- 13 Phases, cancellation, quiescence
- 14 Agent lifecycle
- 15 The tool registry
- 16 The execution pipeline
- 17 Scheduling a step's tool calls
- 18 Approval and sandbox escalation
- 19 Prompt assembly
- 20 Variable interpolation
- 21 Runtime context injection

- 22 Extension points
- 24 Streaming and block assembly
- 25 Failures and retry
- 26 Measuring and spilling
- 27 Pruning and compaction
- 28 Subagents
- 29 Persistence
- 30 Crash repair and chunk packing

Part IV • The engine in motion

- 31 One real turn, annotated
- 32 When things go wrong

Part V • Extending and changing

- 33 Adding a tool
- 34 Composition in full
- 35 Limits and fragile areas
- 36 Design decisions

Part VI • Rebuild it

- 37 A milestone path

Appendices

A • Glossary

B • Data-structure reference

C • Configuration reference

D • Mechanism index

Evidence

Appendix E • Why each mechanism is more than its minimal version

Model I/O (§23–25)

How to read this

This is the digest of a 294-page book, compressed roughly 4:1. It is a **reference**, not a tutorial: it assumes you will read it beside the source, and every claim carries the `path.ts:LINE` that supports it.

What is kept, and what was cut

Kept in full — nothing removed:

- every **citation** (`path.ts:LINE`) — ~600 of them
- every **control-decision table**: the exact branch conditions and where each lives
- every **finding**: what is mounted, what is inert, what is dead, what the docs get wrong
- every **configuration default**, resolved across all three composition layers
- every **limit and fragile area**, including the acknowledged in-source TODOs
- the **design rationale** — §E is the consolidated "what the real implementation adds and which failure each addition prevents"
- 21 of 27 **diagrams**

Cut — the ~20%:

- **exercises** (~120 of them) and the "what you'll learn / prerequisites" apparatus
- **narrative scaffolding** — the extended problem statements and analogies that open each full chapter
- **long code excerpts**, trimmed to the lines that carry the decision
- **repetition** — the three-layer composition hazard is explained once here rather than at each site
- 6 diagrams whose content is stated more compactly as a table

Reach for the full edition when you want the reasoning developed at length, worked exercises, or the complete ~25-line excerpts. Everything factual is here.

Reading paths

- **Orient in twenty minutes:** §1 → §2 → §4, then §31 (one real turn, annotated line by line).
- **Understand the core:** §5 → §6 → §7 → §9 → §10 → §11. Those six are the thesis and its proof.
- **Debug something specific:** Appendix D maps mechanisms, files, and extension points to sections; §32 traces three failure paths end to end; §35 lists where the guarantees stop.
- **Rebuild it:** §37, then the sections each milestone names. Milestones 1–7 need no plugin framework.

Evidence

Written entirely from reading the implementation, then checked by building and running the suite: **3,373 of 3,383 tests pass**, including **342/342** covering the engine core. Every failure on the test machine traces to a Windows limitation rather than to behavior. Marks used inline: 🟢 run-verified · ⚠️ a docs-vs-code contradiction or an unverified quoted figure · ❗ a confirmed limitation · ✅ a specifically verified negative ("nothing does this").

Three claims in the first draft were **wrong** and are corrected here; §E's closing note explains how each was caught, because the failure mode is instructive.

Part I • Orientation

1 The core idea

An agent loop talks to a model, runs the tools it asks for, feeds results back, repeats. What makes this engine worth studying is **where it keeps state**: not an in-memory transcript, but an **append-only event log**, from which every request is *recomputed* immediately before dispatch. Nothing is overwritten or deleted.

That one decision collapses four hard problems into one operation: resume reads the log, fork copies a prefix, trimming appends a "these are superseded by this summary" marker, replay re-folds a recorded log. The rule is repo-wide — *"Model-visible $\Leftrightarrow$ logged: anything that reaches a model request must be reconstructable from the session log"* (`AGENTS.md`) — and is stated as a runnable assertion whose failure message is **"log-reconstruction desync"** (§11).

Three consequences run through everything:

- **Writing to the log is how you change anything.** No side channels.
- **History is rewritten by addition.** A summary plus a marker; the raw events stay forever (§6).
- **The loop is small and not clever.** 1,756 lines. It does not know how to retry, compact, or delegate — those are plugins on documented extension points.

2 Just enough architecture

One application entry, `apps/server`, hardcoding `profile: 'web'` (`src/index.ts:16`). `--profile` is refused outright:

```
if (argv.some(arg => arg === '--profile' || arg.startsWith('--profile='))) {
  program.error('error: this application serves the Web GUI only; --profile is not supported')
}
```

— `apps/server/src/args.ts:33–35`

A profile is not a file. It is an empty root config — `prepareProfile` writes the literal `[]` (`profile-boot.ts:80–84`) — plus an ordered stack of patch layers. No file in the repo lists the web composition (§34).

Cordis, the vendored plugin framework, contributes four concepts:

Concept	Meaning
Context	Reading an unknown property is a <i>service lookup</i> walking the fiber chain; throws if unregistered (<code>reflect.ts:152–166</code>)
Service	Construction is registration — <code>Service</code> 's constructor calls <code>provide()</code> before your body runs (<code>service.ts:42–59</code>)
Fiber	One plugin instance. Stays <code>PENDING</code> until every <code>inject</code> dependency resolves; unloads automatically when one disappears (<code>reflect.ts:314–336</code>)
Effect	<code>ctx.effect(fn, label)</code> runs <code>fn</code> and collects its return as teardown. <i>Every</i> registration in the codebase is one

One nuance that matters later: disposers nested inside one effect run strict LIFO, but **sibling top-level effects on a fiber tear down concurrently** (`fiber.ts:675–696`).

The engine is one row among ~90:

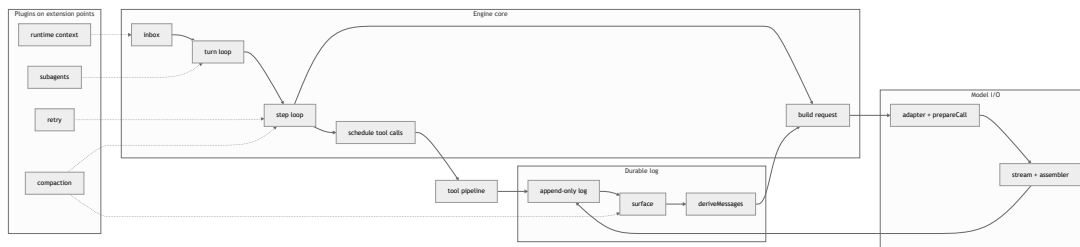
```

- id: agent-loop
  name: '@deepseek-ai/dsh-agent-loop'
  config: { agents: [] }

```

— packages/bundle/base/cordis.patch.yml:486-489

It injects exactly six services — `agents`, `sessions`, `llm`, `tools`, `systemPrompt`, `sessionProjections` (`index.ts:353`). Everything else reaches it through extension points; the engine does not know those plugins exist.



Solid arrows are the main path of a turn. **Dotted arrows are plugins reaching in** — and they are the interesting part: compaction rewrites the surface and can force a request to be re-issued, retry decides whether a failed request runs again, and runtime context injects through the inbox so it becomes ordinary logged history rather than a side channel.

What is *not* running

A recurring hazard: capable-looking packages that never mount.

Thing	Status
Hook bridges (<code>hooks-claude-code</code> , <code>hooks-codex</code>)	absent from every composition layer
<code>dsh-schedule</code> , <code>ui-schedule</code>	opt-in overlay; UI row <code>disabled: true</code>
<code>packages/experimental/**</code> (106 files)	referenced by no composition file
Runtime invariants (<code>dsh-invariants</code>)	mounted nowhere (§11)
SQLite session search	mounted <code>openAt: never</code> — never opened
Native DeepSeek adapter	<code>disabled: true</code> in web (§32)
<code>tool-str-replace-editor</code>	disabled in web, not re-mounted by the preset
<code>skill-badge</code> , <code>hmr</code>	<code>disabled: true</code> in base
<code>agent/turn-stopping</code> listeners	point is live; zero live listeners
<code>tools/pre-execute</code> producing ask	seam complete; no always-on producer

3 Core data structures

Examples come from `snapshots/session/text-turn/`, a recorded session. **It is normalized** — `{{session:1}}`, `{{cwd}}`, `{{message:N}}`, `{{system}}`, `{{tools}}`. The identity tokens are *relationship-preserving*: numbered by first appearance and reused, so the same token twice proves the same id (`identity.ts:64`).

`SessionEvent<T>` (`types.ts:391-398`): `seq · time · type · data`, plus optional `ignorable?: true`, and `surfaceOp` / `sourceEventSeqs` on surface-eligible types only.

seq is the log index, assigned at one site — `seq: this.log.length` (`index.ts:627`) — and re-verified at three more.

`SessionEventMap` core entries (`types.ts:216-320`):

Type	Payload
turn/start · turn/end	{turn} · {turn, reason: TurnEndReason}
step/start · step/end	{turn, step}
user/message	UserMessage — the data is the message
assistant/chunk	{turn, step, chunk: StreamChunk}
assistant/message	{turn, step, message, usage?, interrupted?}
tool/call	{turn, step, callId, name, arguments}
tool/result	{turn, step, message, error?, meta?}
request/header	{header: EpochHeader, reason, startsSeries?}
request/context	RequestContext

The map is open; the build generates the full ~50-type vocabulary into `known-event-types.ts:22-74`, which is a **read-side gate**: a stored log containing a type outside it is refused unless the event is `ignorable`. That is why adding an event type needs no format bump. `SESSION_FORMAT_VERSION = 0` (`types.ts:51`) moves only for structural change.

TurnEndReason (`types.ts:150-169`): `completed` · `max-tokens` (**sticky**) · `blocked` · `aborted{reason}` · `error{failure}` · `interrupted` (**only** written by crash repair).

The surface. Only three types may join — `user/message`, `assistant/message`, `tool/result` (`types.ts:330-334`) — and each **must** declare:

```
type SurfaceOp = 'append' | { op: 'replace'; start: number; end: number }
```

— `types.ts:359-361`

```
interface SessionSurface { readonly nodes: readonly number[]; readonly replaceGeneration: number }
```

— `surface.ts:137-142`

Messages (`message.ts:131-140`): `id` · `role` · `content: ContentBlock[]` · `source`. Source kinds: `user`, `plugin` (carries plugin name), `model` (provider/model), `tool` (`callId`). Built only through frozen factories.

Content blocks (`types.ts:54-110`): `text` · `reasoning` · `image` · `tool-call {id, name, arguments}` — arguments a **raw JSON string** — `tool-result`.

A real assistant message, from the fixture (`session.jsonl:23`) — two blocks, reasoning then answer:

```
{
  "role": "assistant",
  "content": [
    {
      "type": "reasoning",
      "text": "The user wants me to reply with exactly the word\n\"PONG\"...",
    },
    {
      "type": "text",
      "text": "PONG"
    }
  ],
  "source": {
    "kind": "model",
    "provider": "deepseek-official",
    "model": "deepseek-v4-flash"
  }
}
```

with real usage `{ "inputTokens": 3091, "outputTokens": 23, "cacheReadTokens": 0, "reasoningTokens": 20 }` — 3,091 input tokens for a one-word answer, almost all of it system prompt and tool schemas (§31).

StreamChunk (`types.ts:364-376`): `block-start` · `text-delta` · `reasoning-delta` · `tool-call-delta` · `block-end` · `usage` · `finish`. `FinishReason: stop` · `tool-calls` · `max-tokens` · `aborted{failure}` · `error{failure}`.

EpochHeader (`types.ts:179-188`): `config: LlmCallConfig` · `adapterDefaults?` · `system?` · `tools?`. `adapterDefaults` is `{ reasoningEffort?: true, maxTokens?: true }` — a record of which fields the *adapter* supplied, so the next request can strip and re-resolve them (§10).

Tools (tools/src/index.ts:214–280 , :549–573):

```
interface ToolDefinition extends ToolSchema {
  output: { schema, render(args, value), presentationMeta? }
  execute(args, exec): Promise<unknown> // returns a VALUE, not prose
  finalizeContent? · timeoutMs? · isConcurrencySafe?(args) · presentCall? · presentResult?
}
type ToolExecutionResult =
  | { isError: false; value: JsonValue; content; meta?; additionalContexts?; concludesTurn? }
  | { isError: true; error: ToolFailure; content; meta?; additionalContexts? }
```

Three facts: **value never reaches the log**; `additionalContexts` are messages for the *next* step; `concludesTurn` is typed `never` on failures, so a failed call can never stop a turn.

Part II • The mental model

4 The engine in ~100 lines

Real control flow, hard parts removed. Names are real names from `agent-loop/src/agent.ts`.

```
private async kick(): Promise<void> { while (await this.turn()) {} }

private async turn(): Promise<boolean> {
  const turn = ++this.turnNumber
  this.session.append('turn/start', { turn })
  let turnEnds: TurnEndReason | null = null, step = 0
  try {
    while (true) {
      const claimed = this.inbox.claim(step === 0 ? 'next-turn' : 'next-step', turn)
      const assembly = await this.ctx.systemPrompt.assemble({ agent: this, scope: this })
      step += 1
      this.session.append('step/start', { turn, step })
      for (const m of claimed) this.session.append('user/message', m, { surfaceOp: 'append' })
      turnEnds = await this.step(turn, step, assembly)
      this.session.append('step/end', { turn, step })
      if (turnEnds && this.inbox.nextStep.length === 0) break
    }
  } finally {
    this.session.append('turn/end', { turn, reason: turnEnds ?? { kind: 'completed' } })
  }
  return this.inbox.hasPending
}

private async step(turn, step, assembly): Promise<TurnEndReason | null> {
  const request = {
    provider: this.options.provider, model: this.options.model,
    system: renderPrompt(assembly), tools: assembly.tools,
    messages: this.session.deriveMessages(), // not this.messages
  }
  this.session.append('request/header', { header: canonicalHeader(request), reason: 'initial' })

  const assembler = new BlockAssembler(); const chunkSeqs: number[] = []
  for await (const chunk of this.ctx.llm.stream(request)) {
    chunkSeqs.push(this.session.append('assistant/chunk', { turn, step, chunk }).seq)
    assembler.push(chunk)
  }
  const message = createAssistantMessage({ content: assembler.blocks(), source: {...} })
  this.session.append('assistant/message', { turn, step, message },
    { surfaceOp: 'append', sourceEventSeqs: chunkSeqs })

  if (assembler.finish.kind === 'max-tokens') return { kind: 'max-tokens' }
  const toolCalls = message.content.filter(b => b.type === 'tool-call')
  if (toolCalls.length === 0) return { kind: 'completed' }
  for (const call of toolCalls) { /* append tool/call, execute, append tool/result */ }
  return null // null = "no ending yet, run another step"
}
```

Three nested loops: driver over turns, turn over steps, step over request retries. **Input is claimed at boundaries**, never mid-flight. **Boundary events carry no model-visible content** — they exist so the log can be *parsed* (crash repair, UI, step attribution). **Returning `null` is how a turn becomes multi-step.**

Part III • The mechanisms

5 The append-only log

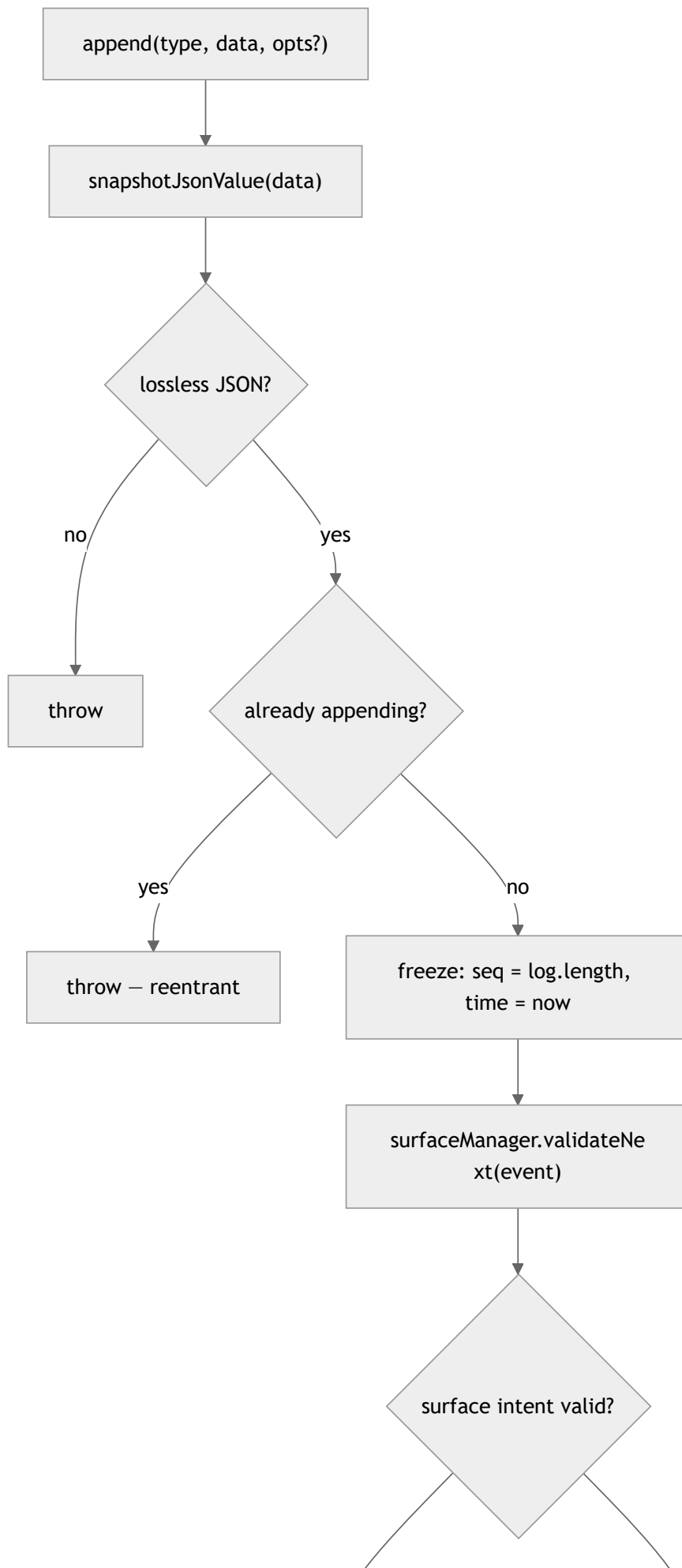
`Session` is a **plain class, not a service** (`core/session/src/index.ts:423`), reached through `Session.create()` or `Session.fromRestore()` (the exclusive-ownership path persistence uses). The Cordis service `SessionStore` (`ctx.sessions`) is **in-memory only** — *"Persistence is intentionally not implemented here — persistence plugins subscribe to `session/event`"* (`:786-789`).

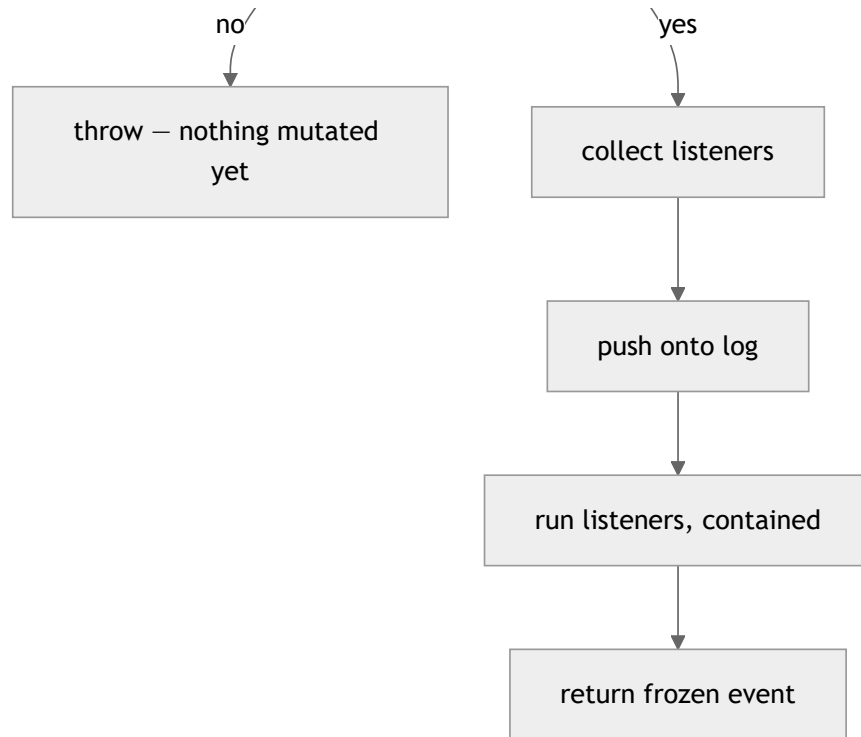
`append<T>(type, data, ...opts)` (`:602-653`). The signature does real work:

```
...opts: T extends SurfaceEventType ? [opts: SurfaceIntent] : []
```

— `:602-606`

A conditional rest parameter: surface intent is **required** for the three surface types and **forbidden** for every other. You cannot forget it, and cannot attach it wrongly.





Snapshot is strict. `snapshotJsonValue` rejects `BigInt`, functions, symbols, `undefined`, `-0`, non-finite numbers, circular refs, sparse arrays, and class instances including `Map/Set/Date` (: 588–591). Anything that would not JSON round-trip identically is refused at the boundary.

Reentrancy is refused (: 621–624) — an append triggered inside another append's synchronous `session/event` dispatch would break `seq = log.length`.

Validate before commit. `validateNext` runs *before* the push, so a bad marker throws with no state mutated. Listeners are snapshotted before the push but run after it (: 636–641); a listener registered during dispatch is not called for that event. Observer failures are contained per listener (: 379–397) — they cannot un-commit.

Decision	Condition	Site
Reject payload	not losslessly JSON	: 612–615
Reject legacy shape	retired <code>request/header-delta</code>	: 616
Reject reentrancy	append in progress	: 621–624
Reject surface intent	range absent / provenance incomplete / wrong type	<code>surface.ts</code> :185–243
Contain listener failure	any throw or rejection	: 379–397

There is no branch where an append is silently dropped: every path commits or throws.

seq contiguity is checked at four boundaries — seed validation `if (snapshot.seq !== index) throw` (: 523–525), the surface fold (`surface.ts`:328–330), persistence batches (`coordinator.ts`:722–726), and the assignment itself. Enforced in one place it is a convention; in four it is load-bearing.

Unknown event types are a hard stop unless skippable:

```

if (KNOWN_SESSION_EVENT_TYPES.has(event.type) || event.ignorable === true) continue
throw this.unsupported(meta, `session "..." contains event type "${event.type}" ...`)

```

— `coordinator.ts`:1143–1148

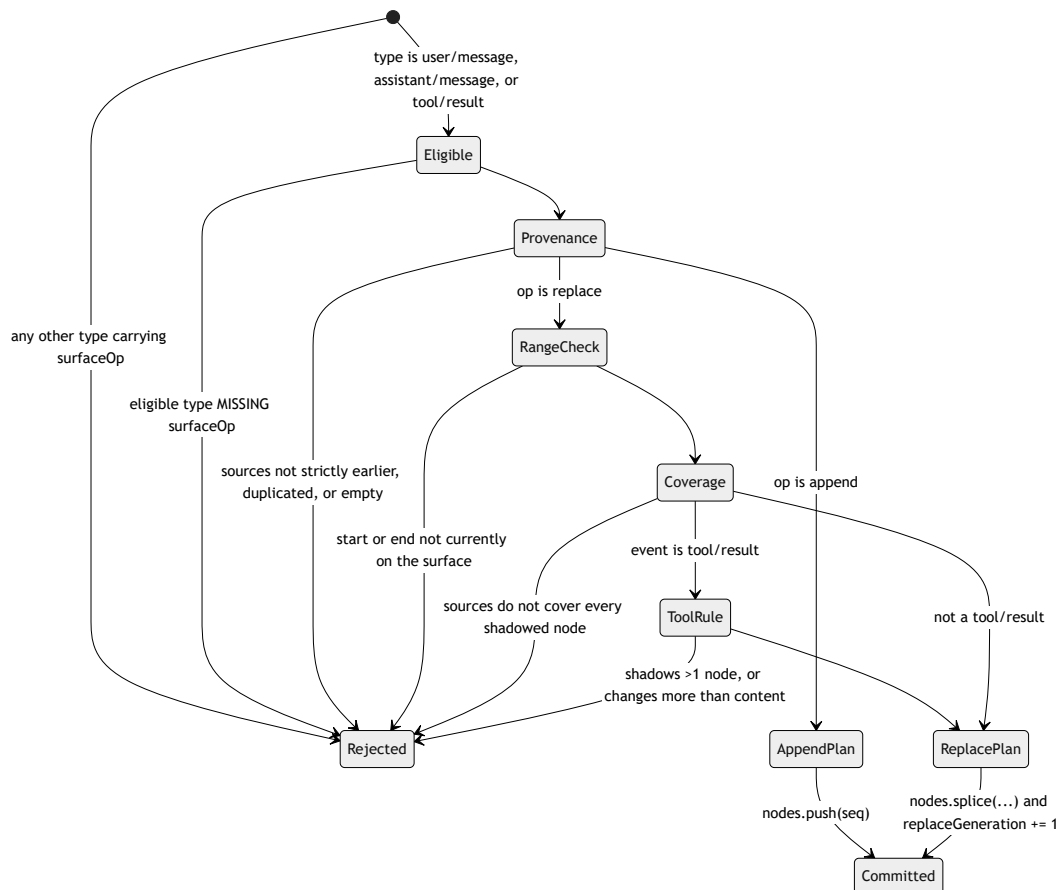
Vocabulary grows freely; a *required* unknown event refuses the session rather than being misread.

The header is deliberately outside the log (`SessionHeader`, `types.ts:56-94`) — "a storage concern, not replayable conversation state." Its `version` is `SESSION_FORMAT_VERSION` (`format.ts:13, 267-276`), and `delegationDepth` is the persisted subagent floor (§28).

No configuration knobs. Everything bounding growth lives outside the log — which is precisely why the log can promise permanence.

6 The surface

The model does not see the log; it sees an ordered list of visible log positions. To "remove" a range you write a new event and rewrite the reading list — the originals stay.



Eligibility is a closed union — plugins cannot extend it. `surfaceOpOf` (`:185-208`) throws in both directions: a non-eligible event carrying surface fields, or an eligible one missing `surfaceOp`. No default.

Provenance (`assertProvenance`, `:210-243`): entries must be non-negative safe integers **strictly earlier** than the citing event's own `seq`, no duplicates, non-empty — *except* an `assistant/message` may carry an empty array, for a known empty provider stream (`:221-222`).

The coverage rule makes a rewrite honest:

```
shadowedSeqs.filter(seq => !sources.has(seq)) // must be empty
```

— `:239-242`

`sourceEventSeqs` must be a **superset of every shadowed node**. You cannot supersede an event without citing it, so a replacement is always reconstructable without guessing from range bounds. (§21 depends on this being complete rather than best-effort.)

Applying:

```
state.nodes.splice(plan.startIdx, plan.endIdx - plan.startIdx + 1, plan.seq)
state.replaceGeneration += 1
```

— :362-379

`replaceGeneration` is incremented **only** here, and is read by four other mechanisms (§7, §10, §21, §27).

tool/result rewrites are deliberately narrow (`assertToolResultRewrite` , :287-318): exactly one shadowed node, and only the tool-result block's `content` may differ — everything else must be deep-JSON-equal, checked by nulling `content` on both sides. This exists for the pruner (§27): it is structurally impossible for pruning to change *whether a tool failed* or *which call it belonged to*.

The surface is the wrong source for a human transcript:

"a landed replacement would erase conversation the user already saw. Append-origin events are that transcript's durable source material; replacement copies stay model-only." — :44-48

Hence `isAppendSurfaceEvent` vs `isReplacementSurfaceEvent` (:35-68). A UI reads append-origin events from the log; the model reads the surface. Same log, two views.

`SurfaceManager` folds incrementally with a `_pendingPlan` cache; `foldSurface(events)` (:387-395) replays a complete log purely. The module avoids `node`: imports so browsers fold the same surface from the same events (:1-9).

Real production writers of `replace`: `compaction` (`compaction-basic/src/region.ts:472-475`) and the tool-result pruner (`compaction-tool-result-pruner/src/index.ts:167-173`).

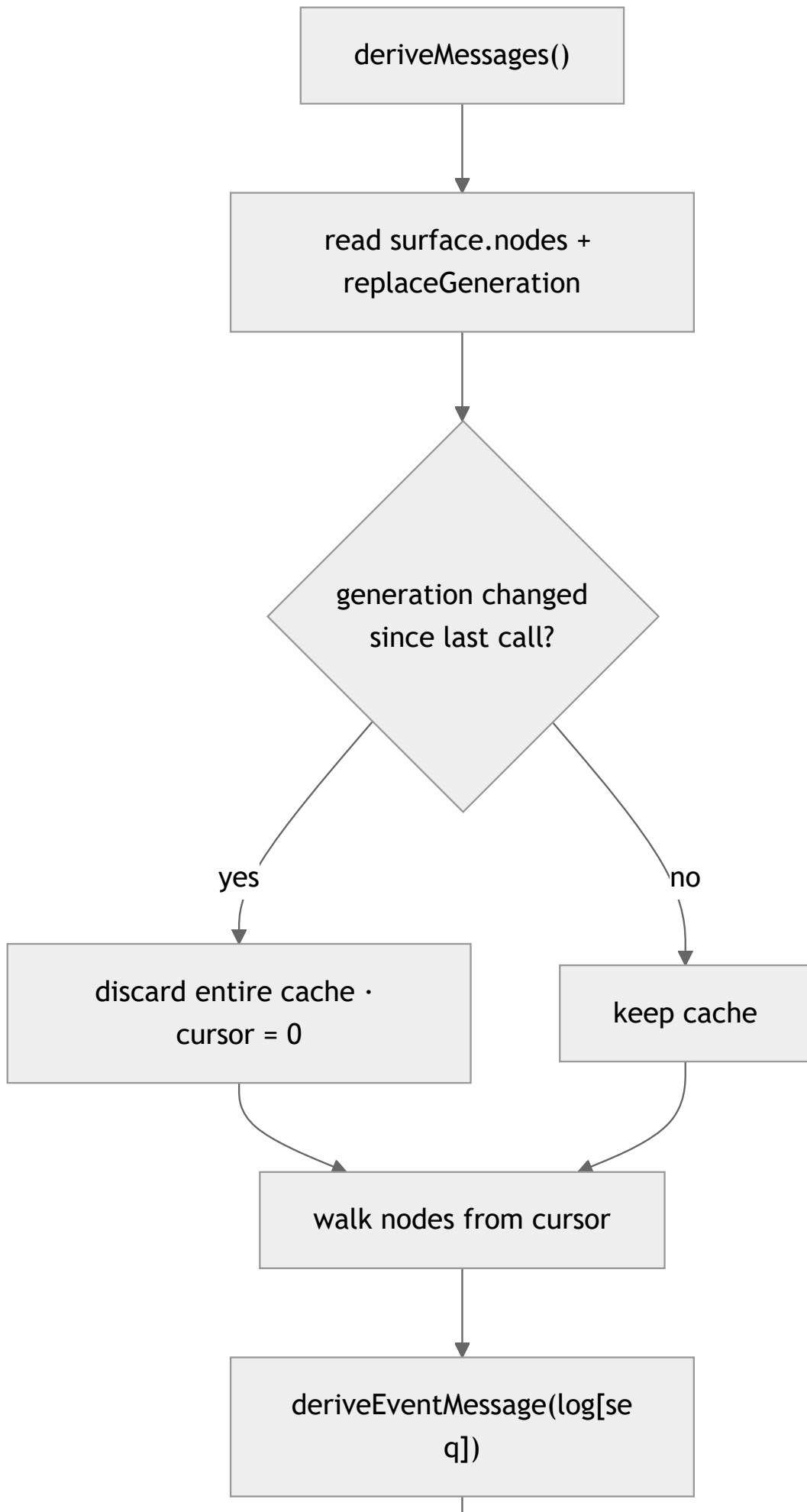
7 From log to request

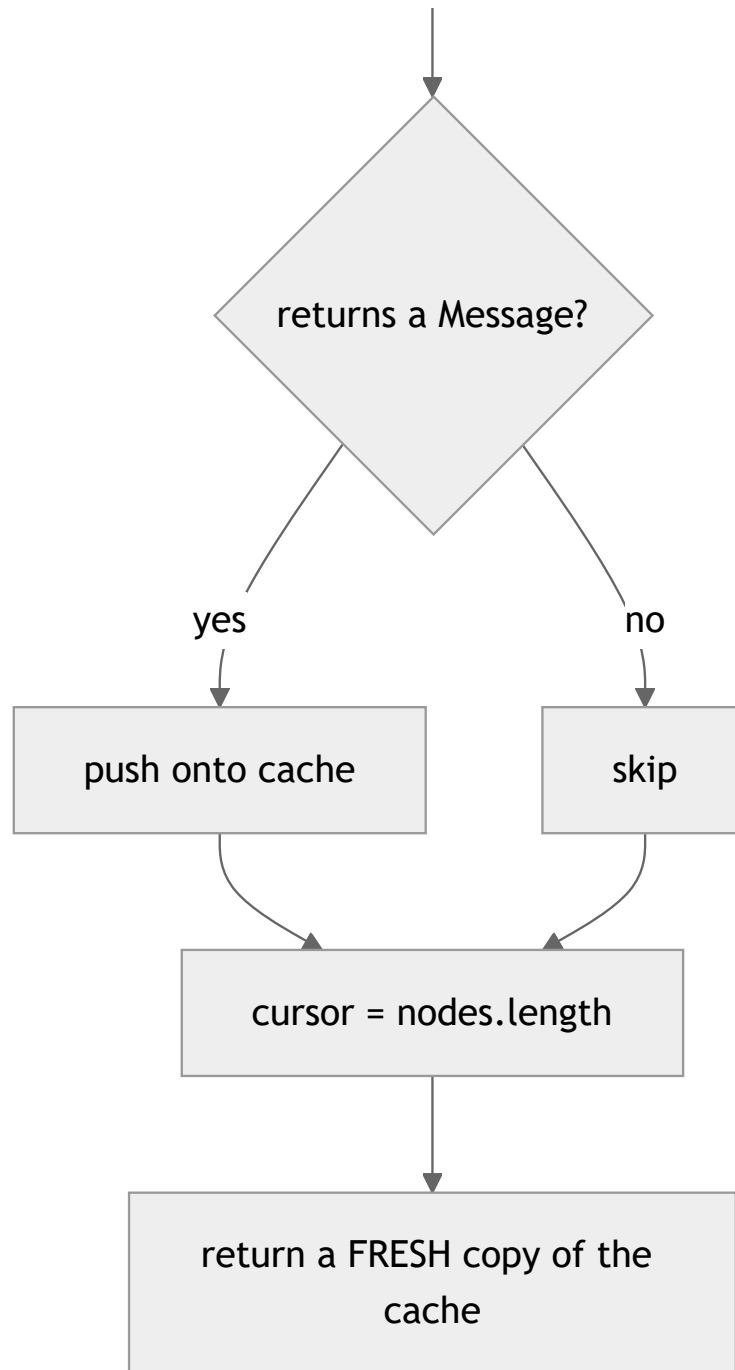
The projection is deliberately dumber than you would expect, because it runs on every request *including replays of old logs* — cleverness here changes the meaning of history recorded under the old behavior.

```
export function deriveEventMessage(event: SessionEvent): Message | null {
  switch (event.type) {
    case 'user/message':      return event.data
    case 'assistant/message': return event.data.message.content.length === 0 ? null :
event.data.message
    case 'tool/result':       return event.data.message
    default:                  return null
  }
}
```

— `surface.ts:83-114`

- `user/message` is a **verbatim pass-through** — "Do NOT re-add per-type framing (e.g. `<context>`) here: framing is caller-owned" (:90-94). What the model sees is exactly what is in the log.
- An **empty-content assistant message derives to null** — a max-tokens step still logs one to carry usage, and it must not enter the transcript (`types.ts:257`).
- The `default` is **intentionally non-exhaustive**, no `assertNever`: "only message-producing events derive history; turn/step boundaries, chunks, usage, and errors are trace/replay data" (:84-86). New event types correctly contribute nothing.





The walker (`index.ts:724-745`) keeps an **append-only cache with a cursor**, and throws the whole cache away when `replaceGeneration` changes — safe only because the surface is otherwise append-only; a replace splices the middle. Cost: $O(\text{new nodes})$ amortized, $O(\text{surface})$ once after a compaction. It returns a **fresh array of shared frozen messages** — no caller can observe another's mutation, and no second deep clone is needed (`:718-721`).

Sole production caller: `agent.ts:355` , feeding the request's `messages` .

8 Projections

The general form of what `deriveMessages` does for messages: a **registered, versioned, pure fold** over the log.

```
interface ProjectionDefinition<K, S> {
  key: K; stateSchema: ZodType<S>
  init(header): S; apply(state, event): S
  wire?: { viewSchema; view(state) }; stateVersion: number
}
```

— session-projection/src/index.ts:42–86

Constraints, all stated in source: `apply` is **pure and synchronous** (*"an async unit would tear the carriers' consistency cut"*, :39); it **must return the same reference when uninterested** — `Object.is` equality "produces zero downstream work" (:58–59); state must be plain JSON, because it is persisted.

`stateVersion` is the cache-invalidation lever: bump it "so persisted (`sessionId`, `key`, `ver`, `seq`, `val`) rows from an older unit are discarded instead of being forward-applied into garbage" (:80–84).

Registration goes through `ctx.effect` (so an unloaded plugin's key vanishes and clients read it as capability absence) and **refcounts** — the same package under N presets registers N times, surviving until the last unmounts; a `stateVersion` mismatch between registrants **throws** (:264–278).

Driving is a double debounce (:625–661): run `apply`; if the reference is unchanged, stop. Only if changed *and* someone is listening, compute the wire `view()`; compare *that* by `Object.is` too; notify only on a real view change.

Worked example — the engine's own projection (`agent-loop/src/index.ts:55–93`, registered :409), key: `'turnBoundary'`, `stateVersion: 2`, no wire (host-only):

```
apply: (state, event) => {
  switch (event.type) {
    case 'turn/start': return { ...state, openTurnStartSeq: event.seq, lastTurn: event.data.turn }
    case 'turn/end':   return { ...state, openTurnStartSeq: null }
    case 'step/start': return { ...state, lastStepStartSeq: event.seq,
                                lastStepBoundary: { kind: 'start', seq: event.seq } }
    case 'step/end':   return { ...state, lastStepBoundary: { kind: 'end', seq: event.seq } }
    default:           return state // same reference – zero downstream work
  }
}
```

Its purpose: a resumed agent continues at turn 8 rather than 1 —

```
const lastTurn = this.loopCtx.sessionProjections.stateOf(session, 'turnBoundary')?.lastTurn ?? 0
```

— agent.ts:101

No counter is stored anywhere; the number is recomputed, exactly like the messages. `openTurnStartSeq` is also how a reader spots a turn left open by a crash (§30).

Cold restore is guarded: `restoreFloor` (:413–423) returns the lowest usable watermark **minus one**, so a suffix read can detect a log that shrank below a stale checkpoint (crash truncation) instead of trusting it; an unusable row with `baseSeq > 0` **throws**, forcing a re-read from seq 0. Checkpoints are `structuredClone d` — "never the live cell reference."

The persisted cache is `session-projection-cache`, injecting [`'storageDomain'`, `'sessionProjections'`, `'sessions'`] (:78), one record per session on the `session_projcache` domain over the JSON backend.

`writeEveryEvents: 200` / `writeIntervalMs: 5000` are throttle bounds *between mandatory checkpoint points*, not the only writes. Reads are synchronous from in-memory tables — never stale-but-wrong, at worst older than the last durable write.

9 The turn and step loops

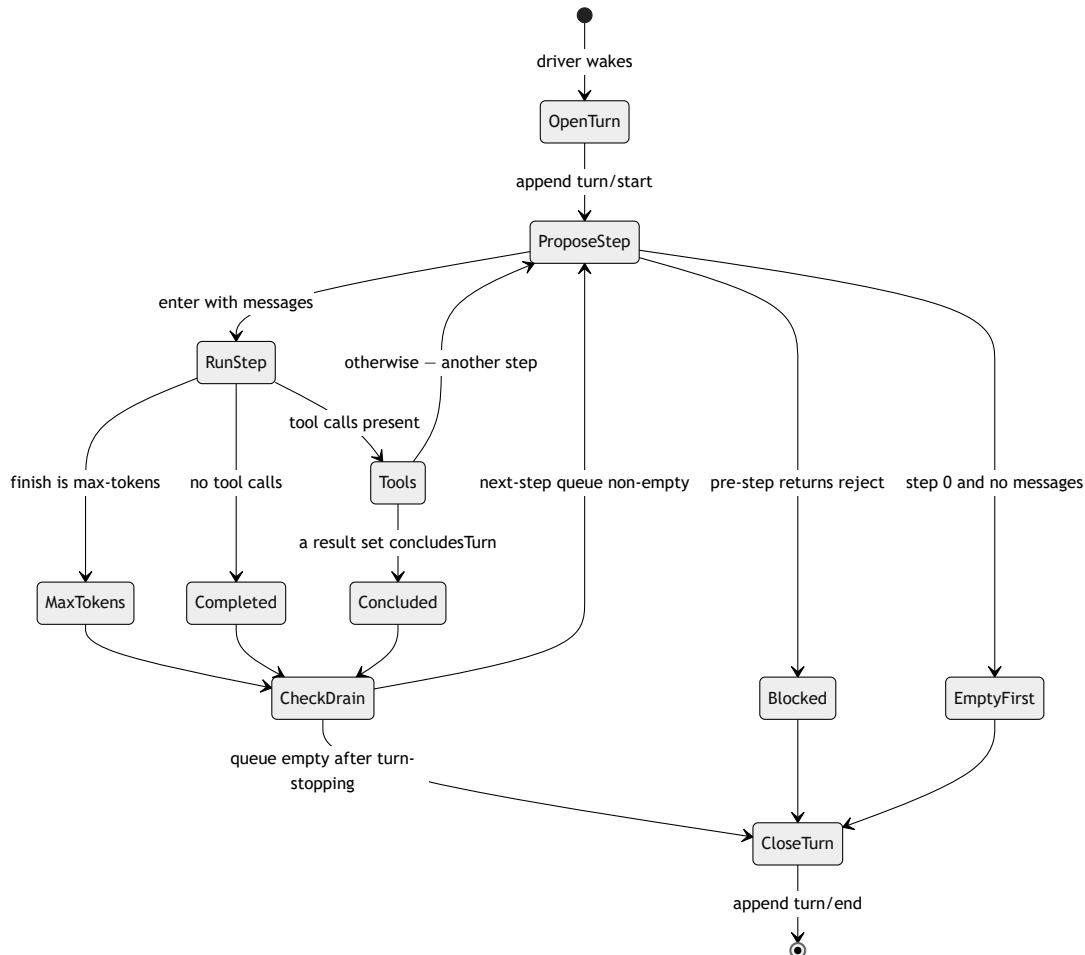
Loop	Site	Iterates	Exits when
driver	agent.ts:221	turns	turn() returns false
turn	:272	steps	an ending is set and next-step is drained
step	:348	request attempts	returns, or nobody asks for a retry

A *turn* is one unit of conversation; a *step* is one model request within it. A turn with three tool round-trips has four steps.

```
private async kick(): Promise<void> {
  try { while (await this.turn()) {} }
  catch (_error) { /* Reported failures and cancellation are contained at the driver boundary. */ }
  finally {
    if (this.phase.kind === 'running') {
      const { turn, wakeRequested } = this.phase
      this.setPhase({ kind: 'idle', lastTurn: turn })
      if (wakeRequested && this.inbox.hasPending()) this.wakeDriver()
    }
  }
}
```

— :219–232

The empty `catch` is correct: by the time an error arrives it has already been reported — `throwError` (:212–217) emits `agent/error` with turn and step *before* rethrowing. Swallowing here is what stops one failed turn killing the agent.



Turn loop, branch by branch (:255–339)

1. `signal.throwIfAborted()` (:273) — first of six checks in this function.
2. `preStep(target, {turn, step})` (:275).
3. **Rejected** → `turnEnds = {kind:'blocked'}`, `return false` (:276–279). This stops the **driver**, not just the turn — no further turns are attempted.
4. `if (turnEnds && decision.messages.length === 0) break` (:280).
5. **First step, nothing to say** → `{kind:'completed'}`, `return false` (:283–286). A removed waking message or a listener rewriting messages to empty "still owns the initial turn boundary, but it spends no model call." **A turn can open and close without ever contacting the model.**
6. Append `step/start`, then one `user/message` per claimed message (:288–293) — where injected context becomes ordinary logged history.
7. Run the step, then fold the ending:

```
if (turnEnds === null || turnEnds.kind !== 'max-tokens') turnEnds = stepEnd
```

— :299. **max-tokens is sticky**: a later clean step cannot mask a truncated answer.

8. `step/end` appended in a `finally` (:301) — always, including on throw.
9. **Last chance to continue** (:304–308): if `turnEnds` is set and `nextStep` is empty, `await dispatch.serial('agent/turn-stopping', ...)`, then **re-test the same condition**. A listener that wants continuation calls `agent.steer(...)`, putting a message in `nextStep`. Data-driven, so listener order cannot change the outcome. *In the shipped profile this point has **zero** live listeners.*
10. Otherwise `target = 'next-step'` (:309).

Exits. On throw (:311–324): if aborted → `{kind:'aborted', reason}` and rethrow; else the failure is **structured** —

```
error: error instanceof LlmError ? error.failure : { message: errorChain(error), code: 'UNKNOWN' }
```

so the log never holds an unstructured error. `finally` appends `turn/end` (:328), and a failure *to append that* is itself routed through `throwError`. Then:

```
if (!this.inbox.hasPending) return false
phase.abort = new AbortController() // fresh controller per turn
phase.wakeRequested = false          // a latch on the old one is stale
phase.step = 0
return true
```

— :333–338

Step loop, branch by branch (:341–438)

Returns `StepEndReason | null`, where **null means "run another step."**

1. Capture `surface.replaceGeneration` (:349) for series detection.
2. `buildRequest(...)`, then `startsRequestSeries = false` (:360) — only the first attempt can open a series.
3. `const stream = preparedCall?.stream(request) ?? this.loopCtx.llm.stream(request)` (:364).
4. Per chunk (:366–370): abort check, append `assistant/chunk`, collect `seq`, `assembler.push`.
5. **On throw while streaming** (:372–389): *only if* `signal.aborted`, salvage `assembler.interruptedBlocks()`; if non-empty append an `assistant/message` with `interrupted: true` citing the chunks so far. Rethrow regardless — salvaging is not recovering.

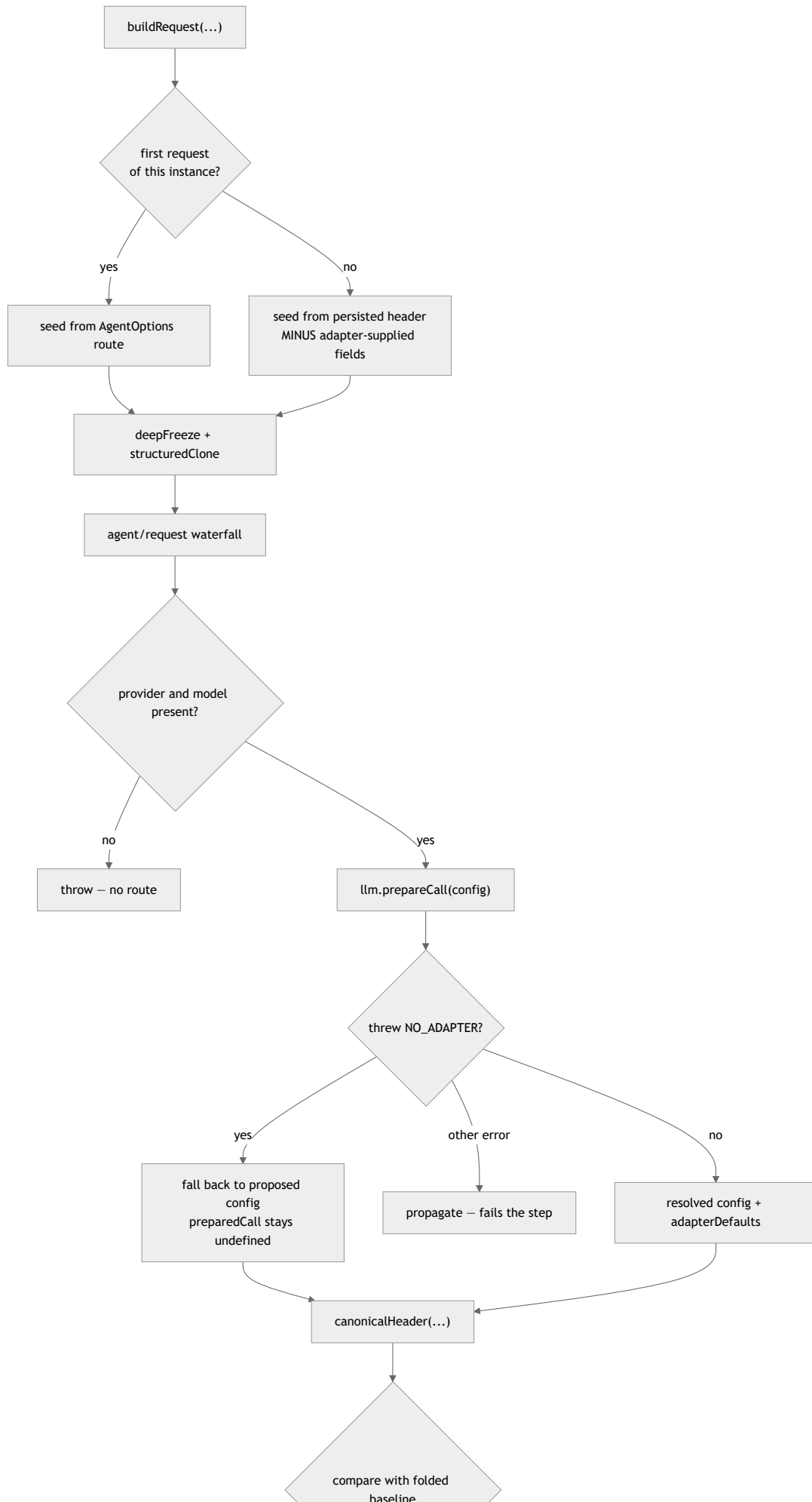
6. **Request failed** (:390-408): run the `agent/request-error` waterfall with default `undefined`. If the action is not `{kind:'retry'}` → `throw new LlmError(...)`. Otherwise `continue`, rebuilding the request from scratch. **The loop never decides to retry.**
7. Append `assistant/message` (:418-427) citing every chunk seq.
8. `finish.kind === 'max-tokens'` → `{kind:'max-tokens'}` (:428).
9. `toolCalls.length === 0` → `{kind:'completed'}` (:430).
10. Else `executeToolCalls(...)`; return `concluded ? {kind:'completed'} : null` (:432-436). The callback passed in splices a tool's `additionalContexts` into the `next-step` queue.

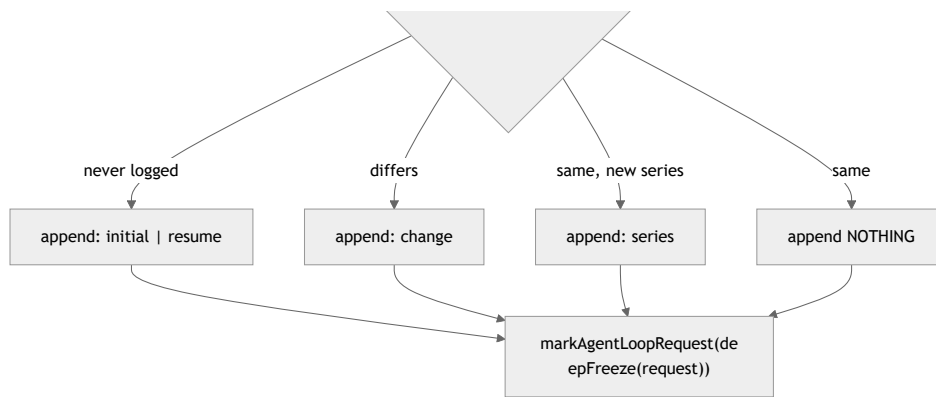
The only condition for a turn to continue: the model emitted ≥ 1 tool call **and** no committed result set `concludesTurn`.

Six abort checks per turn — :261, :273, :287, :303 plus two inside `preStep` — so an aborted turn cannot leave a `step/start` without a `step/end`. **The step retry loop has no cap of its own**; bounding is entirely the listener's job (§25).

10 Building the request

Stage	Behavior
Seed (first request of this loop instance)	<code>{provider, model, reasoningEffort?, maxTokens?}</code> from <code>AgentOptions</code> (:472-476)
Seed (later)	<code>requestProposal(persistedHeader)</code> (:61-67) — strips <code>reasoningEffort / maxTokens</code> where <code>adapterDefaults</code> marks them adapter-supplied, so they re-resolve
Persisted effort	restored only if same provider and same model and not adapter-supplied (:461-465)
Proposal	<code>deepFreeze(structuredClone(seed))</code> , then the <code>agent/request</code> waterfall (:478-481)
Hard fail	proposal lacks provider or model (:483-485)
Resolve	<code>llm.prepareCall</code> ; only <code>NO_ADAPTER</code> falls through to the unresolved config (:488-495) — every other error fails the step





`requestProposal` is the answer to defaults leaking across a model switch: a *user-chosen* `maxTokens` survives, an *adapter-chosen* one does not.

The `agent/request` waterfall is where a `/model` switch lands, and deliberately **cannot touch messages** — *"Model-visible content must use logged channels; this waterfall cannot mutate messages"* (`runtime-types.ts:242–243`).

Header logging is three-way (: 507–518) — the header is logged **only when it changes**, so a thousand-step stable session holds one header event, not a thousand:

Reason	When
<code>initial</code>	first request of this instance, log had no header
<code>resume</code>	first request of this instance over a log that already has headers
<code>change</code>	differs from the folded baseline
<code>series</code>	unchanged, but a new message series began
<i>(nothing)</i>	unchanged and not a series

```
const startsSeries = startsRequestSeries || this.requestSurfaceGeneration !== surfaceGeneration
```

— : 505–506

That second clause is compaction's signature: after a rewrite the next request is not a continuation even though the header is byte-identical. `request/context` (`provider/model/contextWindow`) is likewise appended only on change (: 527–532).

Finally `markAgentLoopRequest(deepFreeze({...}))` (: 535–542) — a process-local `WeakSet` tag (`call-config.ts:66–78`), never serialized, marking *this exact object* as loop-built so a listener can distinguish it from an ad-hoc one-shot call.

11 The reconstruction invariant

35 lines stating the book's thesis as a runnable assertion (`agent-loop/src/invariant.ts:19–55`), attached with two load-bearing flags:

```
ctx.on('llm/stream', (options, next) => {
  if (!isAgentLoopRequest(options)) return next()
  ...
}, { global: true, prepend: true })
```

prepend: true because "a short-circuiting replay listener" registered ahead would otherwise silence it — a check defeatable by registration order is not a check.



Syntax error in text

mermaid version 11.16.0

Checks, in order: request frozen · `sessionId` present · resolves to a live session · `messages` array frozen · log contains a `step/start` · `foldRequestHeader` yields a header · then the central assertion:

```
const expected = session.deriveMessages()
if (JSON.stringify(options.messages) !== JSON.stringify(expected)) {
  fail(`llm request for session "${...}" diverges from the dispatch-time durable derivation (log-reconstruction desync)`)
```

— :39–42

...plus every header field: model, system, temperature, maxTokens, stop, tools (:44–49). Messages come from the surface, the rest from the folded header — together the whole request, which is what makes "reconstructable" complete rather than partial.

It does not run in the shipped system. `dsh-invariants` (the registry that runs companions) appears in **no** `.yaml` anywhere — only `pnpm-lock.yaml`. Its own README: "*dsh-base deliberately omits runtime diagnostics... Loading the registry alone installs no checks.*"

⚠ The same README contradicts itself, claiming in its summary that "the standard agent composition already mounts it." The composition files settle it.

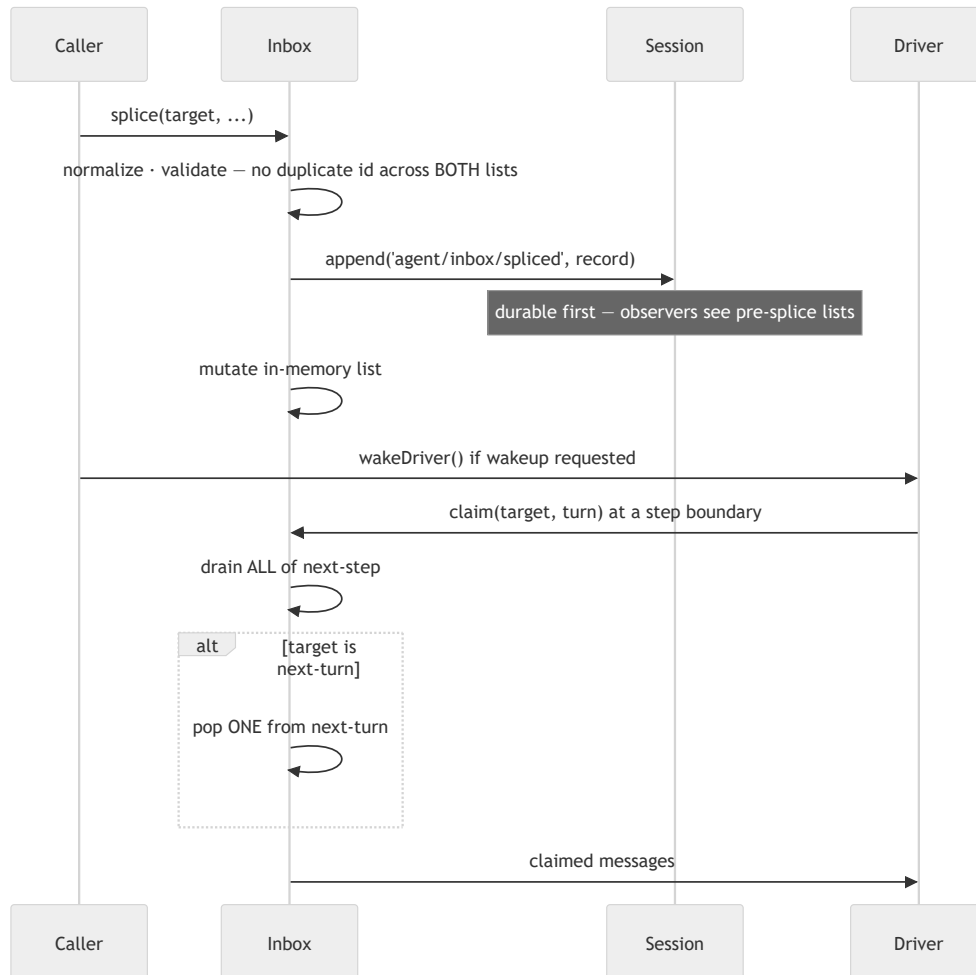
Why it still matters: it is the **executable specification** of what the engine guarantees, and it is the **test oracle** — `request-reconstruction.spec.ts` (24 cases) and `invariant.spec.ts` (8 cases) pass on every run. What ships disabled is runtime enforcement, not verification. Disabling is defensible: it re-derives the whole history and runs two `JSON.stringify` passes per request.

Scope limit, from the package: "*Request reconstruction covers loop-built requests only*" — compaction's one-shot summarization carries a session id and is frozen, and is deliberately **not** checked, because it is not meant to equal the derivation.

12 The inbox

Two ordered queues drained at step boundaries, backed by replayed `agent/inbox/spliced` events — **the queue is the log**, which is why a crashed agent resumes with pending input intact.

```
next-turn: [prompt A] [prompt B]      ← ONE popped per turn opening
next-step: [ctx] [steer] [tool ctx]    ← ALL drained every step
```



Replay starts at `session.header.seedLength ?? 0`, skipping a fork's inherited prefix (already encoded structurally). A malformed persisted splice **throws** (`inbox.ts:34–39`): a queue that cannot be reconstructed exactly is not safe to run on.

Splice is durable-first (`:157–193`): normalize like `Array.prototype.splice`; validate that no message `id` appears twice **across both lists** (`:202–219`, throws `"message ... is already pending"`); then

```

this.session.append('agent/inbox/spliced', splice) // :186
this.state[target] = ...                          // :187

```

Ordering is deliberate — appending first means synchronous observers "see the pre-splice lists" (`:130–132`).

Claim is asymmetric (`:63–78`, marked `@internal`):

```

const claimed = this.mutate('next-step', 0, this.nextStep.length, [], false)
if (target === 'next-turn') claimed.push(...this.mutate('next-turn', 0, 1, [], false))
for (const message of claimed) this.notifications.claimed(message, turn)

```

All of next-step; **exactly one** of next-turn. The `false` is `discardRemoved` — claimed messages get their own `claimed` notification, because "consumed" and "thrown away" are different facts.

Three senders, two axes (`agent.ts:122–141`):

Method	Queue	Wakes?	Meaning
followup	next-turn	yes	one item becomes one turn
steer	next-step	yes	joins at the next step boundary
inject	next-step	no	available when the agent next thinks; never wakes it

 `inject` on an idle agent produces **no activity at all** — only the splice event is logged, status stays `idle`, zero requests (`tests/agent.spec.ts:31-42`).

The reclassification:

```
const wakingAfterAbort = wakeup && this.phase.kind !== 'idle' && this.phase.abort.signal.aborted
const resolvedTarget = wakingAfterAbort ? 'next-turn' : target
```

— :124-126

A waking message arriving after the current activity aborted cannot join it, so it is retargeted to open a fresh turn. The flag is computed **before** the splice so a reentrant `cancel()` from an observer cannot reclassify it.

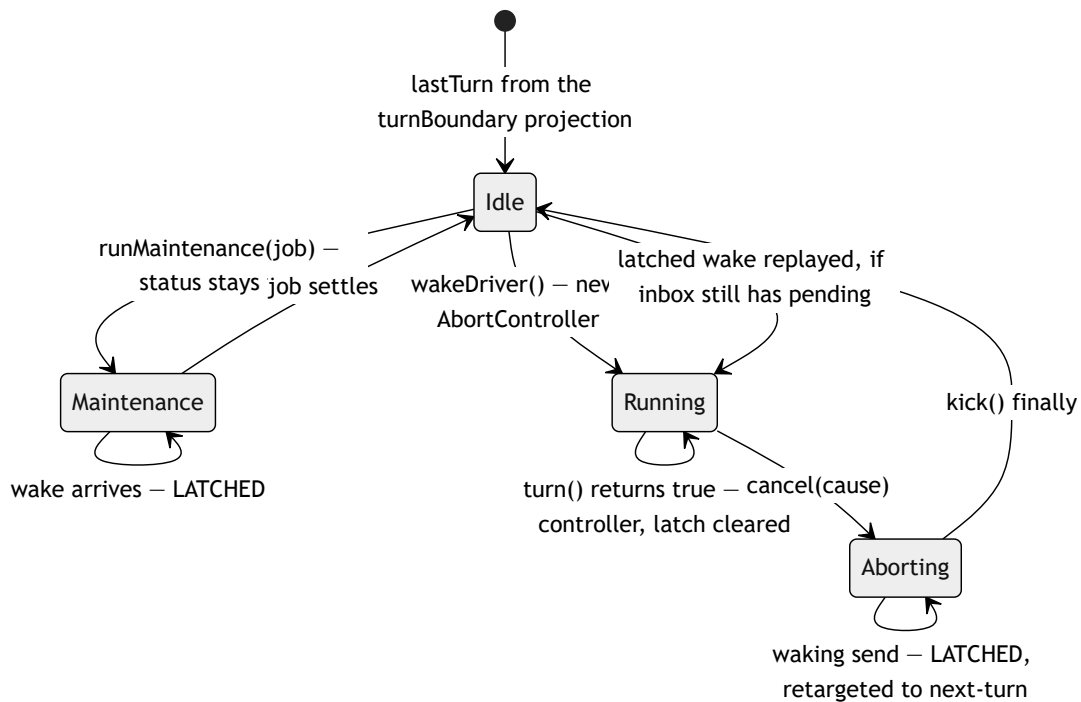
`clear()` (:58-61) discards both queues durably — so cancellation leaves no gap in the record. **A claimed message may never become a user/message**: if the step is then rejected, "the message ends here — never re-emitted" (`runtime-types.ts:196-197`). Consumers must treat `claimed` as consumed, not delivered.

13 Phases, cancellation, quiescence

```
type Phase =
  | { kind: 'idle'; lastTurn }
  | { kind: 'maintenance'; abort; lastTurn; wakeRequested }
  | { kind: 'running'; abort; turn; step; wakeRequested }
```

— :39-47

Externally only two statuses: `maintenance` and `idle` both report `'idle'` (:108-110). Because `setPhase` compares the **collapsed** status (:113-120), an `idle → maintenance → idle` cycle emits nothing — a manual compaction never flickers a UI, and listeners that reset on `idle` are not spuriously triggered.



Cancellation (:143-149) clears the inbox *first* (which also clears the latch — a latch whose message is gone must not start an empty turn), then fires the signal. The `cause` travels: the turn records `{kind: 'aborted', reason}`, so the log says *why*.

The wake latch (:181-202) is the answer to the window where an agent is neither usefully running nor idle:

```

if (reason?.kind !== 'disposed' && (this.phase.kind === 'maintenance' || wakeAfterAbort)) {
  this.phase.wakeRequested = true
}

```

— :186-189

Latched only from **maintenance** or the **aborting window**. A live running driver does not latch — it claims the queue itself. And **disposal never latches**: if it did, teardown would queue a turn while the agent is being destroyed, and `dispose()` awaits `whenIdle()` (§14) — it would wait on a model call it just cancelled.

Replay is guarded identically in two places — `kick()`'s and `runMaintenance`'s *finally*:

```

if (wakeRequested && this.inbox.hasPending) this.wakeDriver()

```

An explicit wake to an *idle* agent always opens its boundary even if its message was cleared; only a **latched replay** is suppressed when the queue no longer holds it (:173-180).

Quiescence:

```

async whenIdle(): Promise<void> {
  let activity: Promise<void>
  do { await (activity = this.activityDone) } while (activity !== this.activityDone)
}

```

— :204-209

The loop is the point: a latched wake replayed in `kick()`'s `finally` installs a *new* `activityDone` before the old one's awaiters run. `whenIdle()` means "no activity, and none started while I waited."

No watchdog. Convergence is entirely cooperative — a tool ignoring its signal delays teardown indefinitely, and there is no timeout or forced kill (§35).

14 Agent lifecycle

The governing pattern: **register the undo before you do**. `prepare()` builds a memoized teardown and registers it with both the factory and the owner fiber **before any resource exists**, over mutable slots filled in as resources appear.

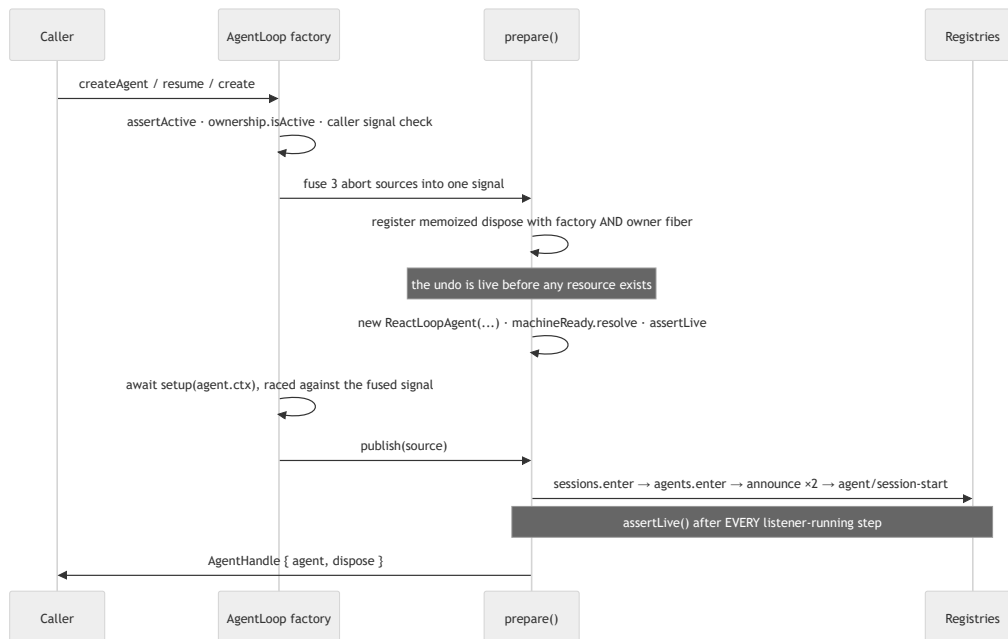
The gate (:523–534): `assertAgentOptions · ownerCtx.fiber.assertActive() · ownership.isActive() · caller` signal not already aborted. `isActive` is both a flag and a fiber-state test — `INACTIVE_STATES = {UNLOADING, DISPOSED, FAILED}` (:38–42) — so a factory whose fiber is unloading cannot own a new lifecycle.

Three cancellations fused into one signal (:542–550), each with its own reason: the caller's, the owner fiber's unload, and factory teardown (which aborts with `new Error('agent loop is not active')`).

The memoized teardown (:560–583):

```
const dispose = (ownerTriggered = false) => (disposing ??= (async () => {
  abort.abort(new Error(`agent "${id}" lifecycle disposed`))
  // ...detach both listeners...
  try {
    if (machine === undefined) await machineReady.promise // disposal racing construction
    if (machine !== undefined) {
      machine.cancel({ kind: 'disposed' }) // the cause that never latches
      await machine.whenIdle()
      await machine.scope.dispose()
    }
  } finally {
    try { detachAgent?(); detachSession?(); }
    finally { untrack(); if (!ownerTriggered) await unfollowOwner() }
  }
})())
```

`disposing ??=` makes it idempotent — every racing owner awaits **one** quiescence. Strict reverse order: stop the machine, wait for it to stop, unwind the scope, leave the registries, release bookkeeping. `ownerTriggered` stops the owner's own effect unregistering itself from inside itself.



Publication (: 619–633) — enter both registries, *then* announce, with `assertLive()` after **every** step that runs listener code:

```

assertLive → sessions.enter → agents.enter → sessions.announce → assertLive
           → agents.announce → assertLive → emit agent/session-start → assertLive
  
```

Each announcement can synchronously run third-party listeners, any of which may start teardown; `agent/created` can even veto publication by throwing synchronously (`agent/src/index.ts:551–567`).

Three entry points, all funnelling through `setupAndPublish` (: 688–708): `create` (fresh, factory-owned), `createAgent` (fresh, caller-owned), `resume` (loaded from disk). The `setup` callback runs **after construction, before publication** — so an agent is never visible without its preset composition (§34) — and is raced against the fused signal so a hanging setup cannot pin the lifecycle.

Resume adds a load barrier (: 747–752):

```

preparation = await raceAbortCall(() => persistence.prepare(id, fused), fused, id,
  (abandoned) => { abandoned[Symbol.dispose]() })
  
```

The fourth argument is `releaseAbandoned` : if the race is lost the load still completes, and the abandoned preparation is **disposed rather than leaked**. Afterwards ownership is re-checked (: 756–757) because everything may have changed while the disk spun.

Factory teardown awaits both live agents and in-flight startup jobs (: 137–145) — an agent still being created when the factory shuts down is awaited too.

Configured-agent edges: `waitForDrainingConfiguredIdentity` (: 494–514) waits on `agent/disposed` + `session/disposed` only if the id currently occupies a registry; restore falls back to fresh creation **only for genuine absence** — corruption and backend failures stay loud (: 479–490); declarative startup failures are *reported* via `agent-loop/config-start-failed` with per-listener try/catch (: 448–467), never thrown.

15 The tool registry

`ToolRuntime` at `ctx.tools` (`core/tools/src/index.ts:780`). A definition separates three things usually conflated:

Concern	Field	Audience
what the model may call	<code>name</code> , <code>description</code> , <code>parameters</code>	the model
what actually happens	<code>execute</code> → a canonical value	the system
what the model reads back	<code>output.render(args, value)</code>	the model

`execute` does not produce prose. It produces data, validated against `output.schema` and *projected* into content. Replay re-renders from the stored value; a UI presenter draws from the same value.

```
register(definition): () => void {
  // output-shape, timeoutMs, reserved-name checks
  return this.layers.effect(this.ctx, layer => layer.tools.insert(name, definition),
    { label: 'tools.register()' })
}
```

— :1028–1053

`this.layers` is a `ScopedLayers` — a global layer plus one per scope — which is what lets one registry serve every agent while a preset-mounted tool is visible only to agents on that preset (§34). Duplicate names in a scope throw (:719–721); the transport name `run_code` is reserved and can never be registered or shadowed (:1045–1047).

`defineTool` (`schema.ts:545–617`) compiles an authoring DSL to JSON Schema and wraps callbacks **asymmetrically**:

```
async execute(args, exec) {                                // STRICT
  const violations = validate(args)
  if (violations.length > 0) throw new ToolArgsError(violations) // every violation, not the first
  return userExecute(args, exec)
}
// presentCall / presentResult / isConcurrencySafe: swallow invalid args → undefined / false (:594–616)
```

The asymmetry is the point: `execute` must be strict because running with bad arguments could do damage; presenters must be lenient because **they run against old logged arguments during replay**, where a schema may since have changed. A presenter that threw would make historical sessions unopenable.

The raw path is live, and instructive. Two production sites register hand-built definitions, both because their schema is only known at runtime: MCP tools arrive from a remote server (`mcp-client/src/tools.ts:182`), and the subagent structured-output tool takes the caller's requested shape (`subagent-in-process-driver/src/structured.ts:74`). The latter re-implements validation by hand — *"ToolArgsError → isError result with INVALID_ARGS: the model retries within the same turn, exactly like a schema-validated defineTool call."* That is the obligation the raw path carries: **the framework validates nothing for you.**

`finalizeContent` is snapshotted at call creation (:1409), so a tool unregistered mid-call still gets *its own* finalizer, not whatever now occupies the name.

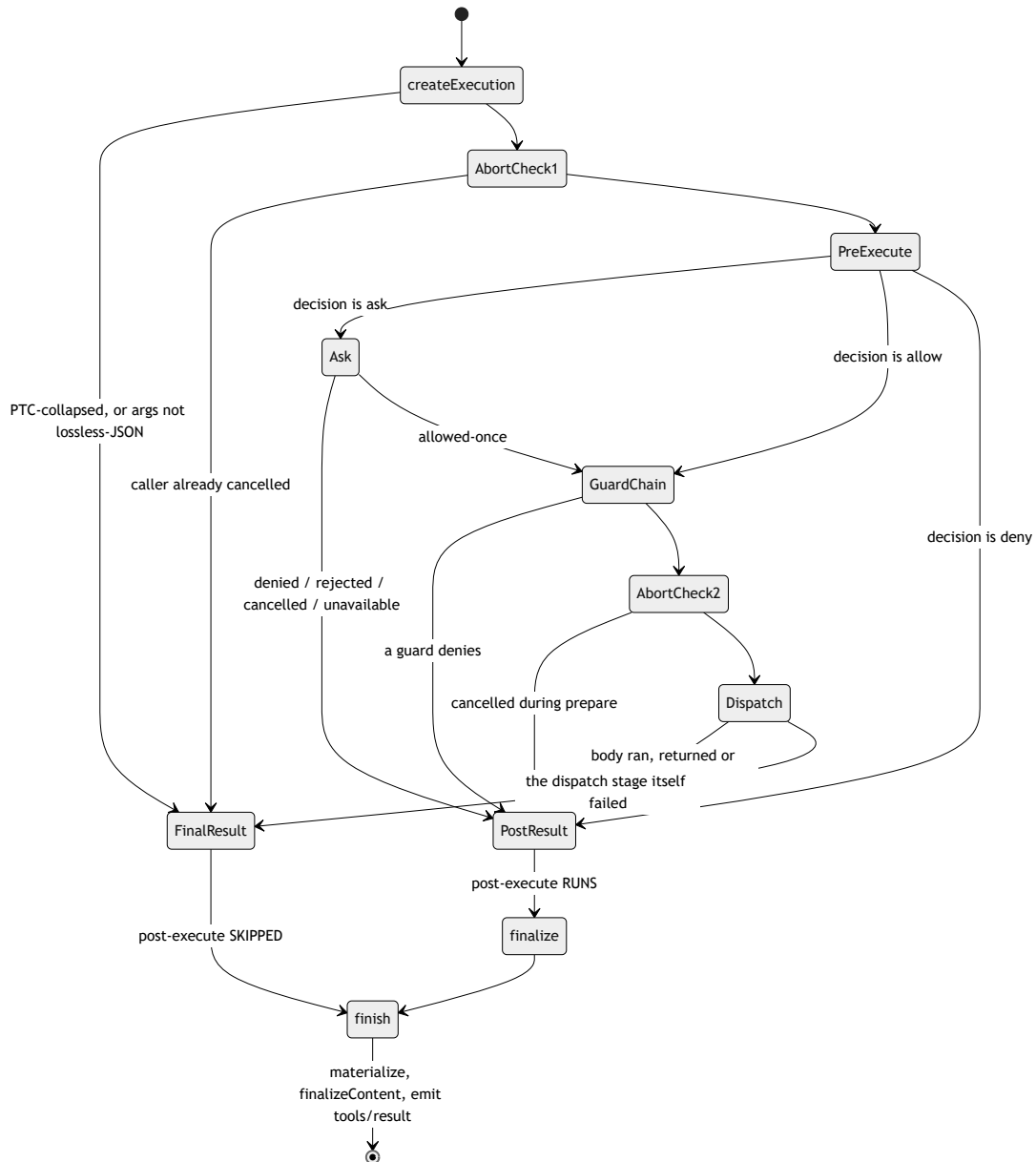
The tool list is part of the request header, so registry changes log a `change` header and invalidate provider prefix caching — which is why plan mode restrains by instruction rather than by removing tools: *"The tool catalog stays the same across modes for request-cache stability... those tools remain listed only to keep the request shape stable"* (`base/cordis.patch.yml:315`).

16 The execution pipeline

```
interface ToolRuntimeScheduler {
  prepare(exec): Promise<ScheduledToolPreparation> // ordered, may await – MAY this run?
  dispatch(exec): Promise<ScheduledToolDispatch> // overlappable – what happened?
  finalize(exec, result): Promise<ToolExecutionResult> // ordered – change it?
  finish(exec, result): ToolExecutionResult // sync – freeze, notify
}
```

— :444–453, reached via the `TOOL_RUNTIME_SCHEDULER` symbol (:459)

The ordinary `execute()` chains the same four (:1333–1353); the scheduler merely exposes the seams so a caller can interleave (§17).



prepare (:1454–1498) checks the **PTC collapse first**, before anything observes the call, because *"pre-execute listeners, approval ask, and guards must never observe — or worse, approve — a call that can only fail"* (:1364–1370). Then: caller-abort → `tools/pre-execute` waterfall (default `{kind: 'allow'}`) → approval if `ask` → guard chain (only if the decision was `allow`) → second abort check → `{kind: 'dispatch'}`.

dispatch runs the `tools/execute` around-waterfall whose innermost `next()` is the body. `fuseToolSignals` (:1880–1907) re-fuses the caller's *original* signal beneath whatever a wrapper substituted — **a wrapper cannot detach caller cancellation**. Cancellation *during* dispatch yields `TOOL_ABORTED` (body ran) rather than `ABORTED_BEFORE_DISPATCH` — the distinction a caller needs to decide about retrying. `dispatch` never returns `'dispatch'`; only `post-result` or `final-result`.

finalize runs `tools/post-execute: block → isError` with the decision's feedback, **discarding tool-deferred context**; `accept` with `value` → re-validated, and it **cannot** replace the value of an already-failed result (`TypeError`, :1756–1758); both `content` and `value` → `TypeError`. **finish** materializes, applies the snapshotted `finalizeContent`, freezes `exec`, and emits `tools/result` with contained listeners.

The `needsPost` rule — decided by which stage produced the result

Outcome	From	Post-execute?
PTC collapse, bad args	<code>createExecution</code>	no
Cancelled before prepare ran	<code>prepare</code> entry	no
Denied by pre-execute or a guard	<code>prepare</code>	yes
Cancelled while approval pending	<code>prepare</code>	yes
Body ran, returned or threw	<code>dispatch</code>	yes
Dispatch stage itself failed	<code>dispatch</code>	no

Denials get post-execute — deliberate: post-execute is about *results*, and a denial is a result (*"Thrown tools still reach this waterfall as errors"*, :420–422). What skips it are calls that never became a real dispatch candidate.

Control decisions

Decision	Condition	Site
Short-circuit to <code>final-result</code>	PTC-collapsed, or arguments not lossless-JSON	:1427–1441
<code>final-result</code> (abort)	caller signal already aborted at entry	:1461–1463
Run approval	pre-execute returned <code>ask</code>	:1470–1472
<code>post-result</code> (abort)	cancelled <i>while</i> approval was pending and the channel reported cancellation	:1474–1476
<code>post-result</code> (denial)	pre-execute <code>deny</code> , or a guard denies an <code>allow</code>	:1477–1490
<code>{kind:'dispatch'}</code>	everything passed	:1494
<code>final-result</code>	anything in <code>prepare</code> threw	:1495–1497
<code>TOOL_ABORTED</code> vs ... <code>_BEFORE_DISPATCH</code>	<code>state.bodyInvoked</code>	:1509–1516
<code>TypeError</code>	post-execute <code>accept</code> supplies both <code>content</code> and <code>value</code> , or replaces a failed result's value	:1748–1758

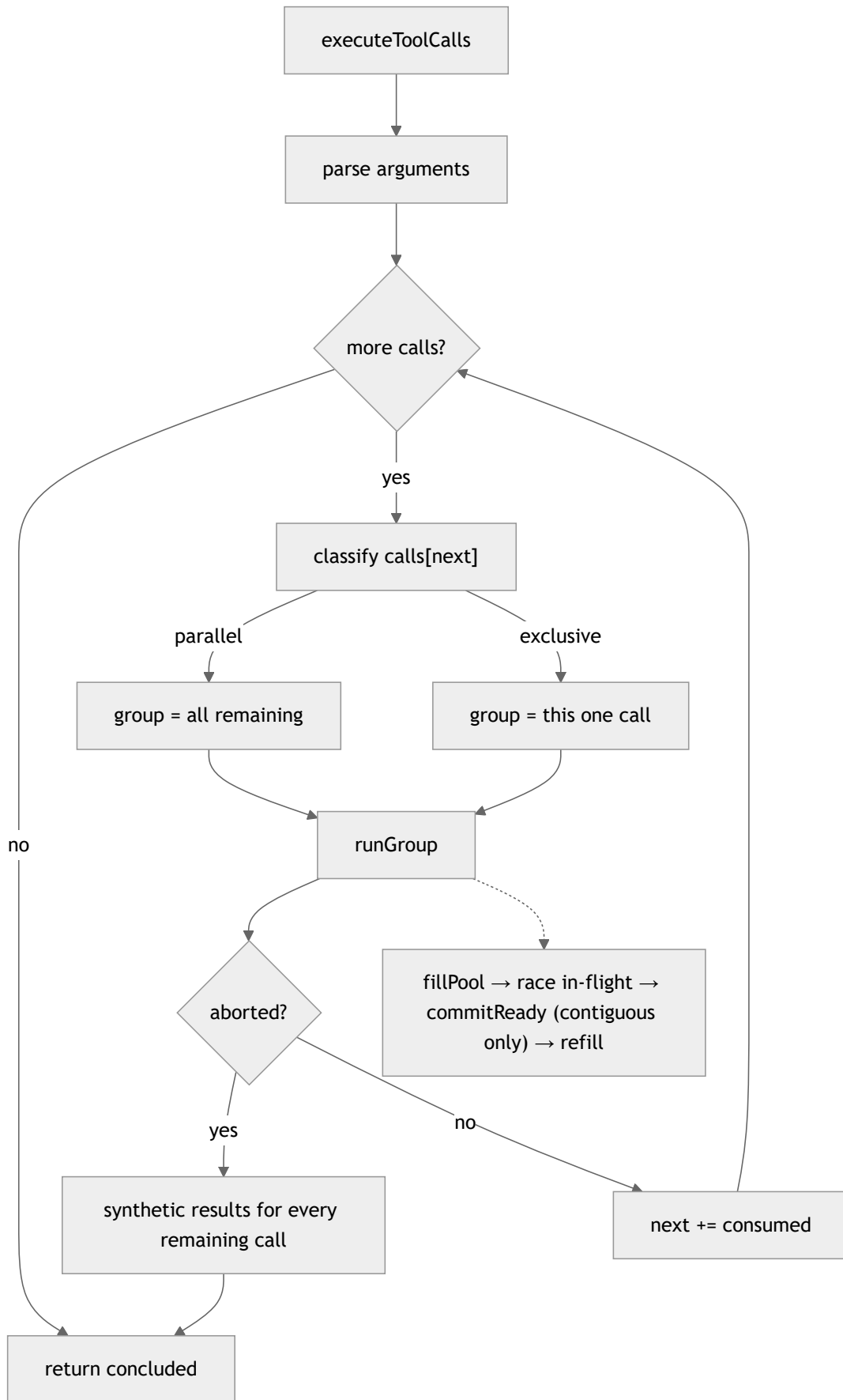
Three execution views matter when writing a wrapper: `ToolExecutionInput` (caller-supplied) → `ToolExecution` (registry-internal; adds `rootCallId` and an opaque `token`, frozen just before `tools/result`) → `ToolRunContext` (handed to `execute`; adds `deferContext` and `concludeTurn`). A fourth,

`ToolDispatchExecution` (:384-387), deliberately makes `signal` **mutable** so an around-dispatch wrapper can substitute its own — while `fuseToolSignals` re-fuses the caller's original underneath regardless.

`run_code` is a **second consumer** of this same interface, re-implementing the ordered-prepare / overlapping-dispatch / ordered-commit discipline for sub-calls (`ptc.ts:280-587`) — which is why the symbol is exported at all.

17 Scheduling a step's tool calls

Three rules: **classify at the boundary** — the first call's mode decides the group (parallel → all remaining; anything else → a one-call **barrier**); **dispatch may overlap, commits may not; every model call gets a result**, including ones cancellation prevented from starting.



```
function parseArguments(raw: string): unknown {  
  try { return raw ? JSON.parse(raw) : {} } catch { return raw }  
}
```

Invalid JSON returns the **raw string**. Not a bug: malformed model output must become a readable tool error, not a crash. It reaches `execute`, where `defineTool` rejects it as `ToolArgsError` — an ordinary `isError` result that still runs post-execute, so the model can correct **within the same turn**.

`executionMode` is synchronous and **fail-closed**: no declaration, a throwing classifier, or any non-`true` return all yield `exclusive` (`tools/index.ts:1267-1276`).

Pool filling (`:199-214`) re-reads modes live — a call classified parallel when the group formed may reclassify before it starts, and the pool stops there, making it the next barrier. A batch that began all-parallel can split mid-flight. The cap is destructured **per group** (`:132`) from a read-through getter, so a settings change caps the *next* group without disturbing one in flight.

Ordered commit (`:147-161`) — `if (slot === undefined) break` is the whole rule: advance only across **contiguous** settled slots. Consequences: `finalize` / `finish` run in model order, and `additionalContexts` reach the next step deterministically even from parallel tools.

The `tool/call` event is appended **before** `prepare` (`:168`), so the log records an attempted call even if policy denies it — and that presence is exactly what lets crash repair distinguish its two cases (§30).

Cancellation vs failure is the best-designed distinction here:

- **Abort** → started calls drain and commit in order, then every unstarted call gets a synthetic `tool/call` + `tool/result` pair with `ABORTED_BEFORE_DISPATCH` (`:250-260`), *"so replay stays valid"* — a cancelled turn is still a well-formed conversation.
- **Scheduler failure** → stop new dispatches, `await Promise.allSettled(inFlight)`, rethrow the first failure **without fabricating results** (`:232-236`). Cancellation is an expected outcome with a defined per-call meaning; a scheduler bug is not, and inventing results would hide it.

`concluded ||= result.concludesTurn === true` (`:158`) — one result ends the turn. Real producer: the subagent structured-output tool (`structured.ts:94`).

Concurrency declarations in shipped tools: `read` (`tool-fs`) and `web_search` declare `() => true`; `bash` / `pwsh` and `str_replace_editor` declare nothing → `exclusive`, **even for the editor's read-only view command**. Two first-party file readers, opposite choices. `maxParallelToolCalls` defaults to `10` (`constants.ts:6`) and is **deployment-wide** (§35).

18 Approval and sandbox escalation

Two independent mechanisms sharing one service; conflating them is the hazard here.

	Pipeline seam	In-body escalation
Where	<code>tools/pre-execute</code> → <code>ask</code> → <code>ApprovalService</code>	inside the tool's own <code>execute</code>
Who triggers	a policy listener	the model , via tool arguments
Asks	may this run at all?	may this run with wider permissions?
Denial becomes	a synthetic <code>post-result</code>	a thrown error → ordinary <code>isError</code>
Live in web?	no producer	yes

```
type PreToolDecision = { kind: 'allow' } | { kind: 'deny'; reason } | { kind: 'ask'; reason? }
```

Approval resolves **inside prepare** — which matters for the scheduler: prompts for a parallel group stay in model order. Degradations are deterministic: no `ApprovalService` composed $\rightarrow$ `deny` ("not yet supported"); no `exec.agent` $\rightarrow$ `deny` (no session to route through). **An unanswerable question is refused, never allowed.**

`ApprovalService.request()` (`user-approval/src/index.ts:222-241`) **refuses outside an open turn** (an audit event belonging to no turn is unreadable), appends `approval/asked` , decides, appends `approval/decided` . Both are durable and **log-only** — never surface-eligible, so the model never sees that a human was asked.

```
if (policy === 'never') return 'rejected'
```

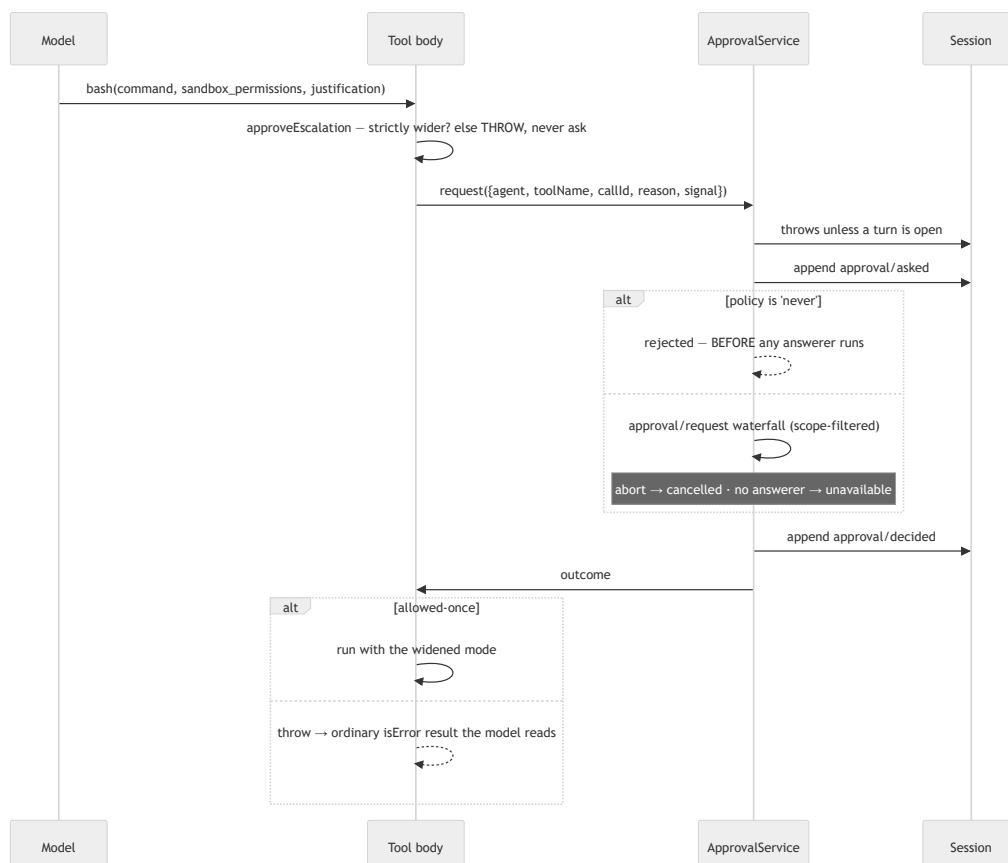
— :277

Checked **inline, before the approval/request waterfall runs at all**, specifically so no `prepend: true` listener can claim the request ahead of a policy check. Registration order cannot leak a grant. (The same insight as §11's `prepend` , applied in the opposite direction.)

✓ **The generic seam has no always-on producer.** Repo-wide, the only non-test source of `{kind: 'ask'}` is `hooks-claude-code/src/index.ts:243` — and the hook bridges are mounted nowhere.

What ships is in-body escalation (`sandbox/src/escalation.ts:157-189`): check the requested mode is *strictly wider* via a `WIDER_MODES` table — **throw, never ask**, if not (prompting for a no-op trains people to click yes); throw if no approver or agent; request; `allowed-once` $\rightarrow$ return the granted mode, every other outcome throws.

The model requests it through **arguments on the same call** (`tool-bash/src/index.ts:329-338`), so the request *and its justification* are durable in the `tool/call` event. The `sandbox_permissions / justification` parameters are **only advertised when something actually confines the tool** (`escalationModes.length > 0` , :192) — never tell the model about an inapplicable capability. A denial propagates out of `execute` , is caught by `dispatchToolBody` (:1545-1546), and becomes a readable `isError` result.



Both policies are also **stated in the prompt** as runtime-context sections (§21). Under policy: 'never' the approval section tells the model *not to ask*, because asking would be auto-rejected — enforcement and instruction kept consistent by living in the same plugin.

19 Prompt assembly

```
interface PromptAssembly {
  sections: AssembledSection[] // → the system prompt
  contexts: AssembledContext[] // → a runtime-context MESSAGE
  tools: ToolSchema[]
  variables: Record<string, string | undefined>
}
```

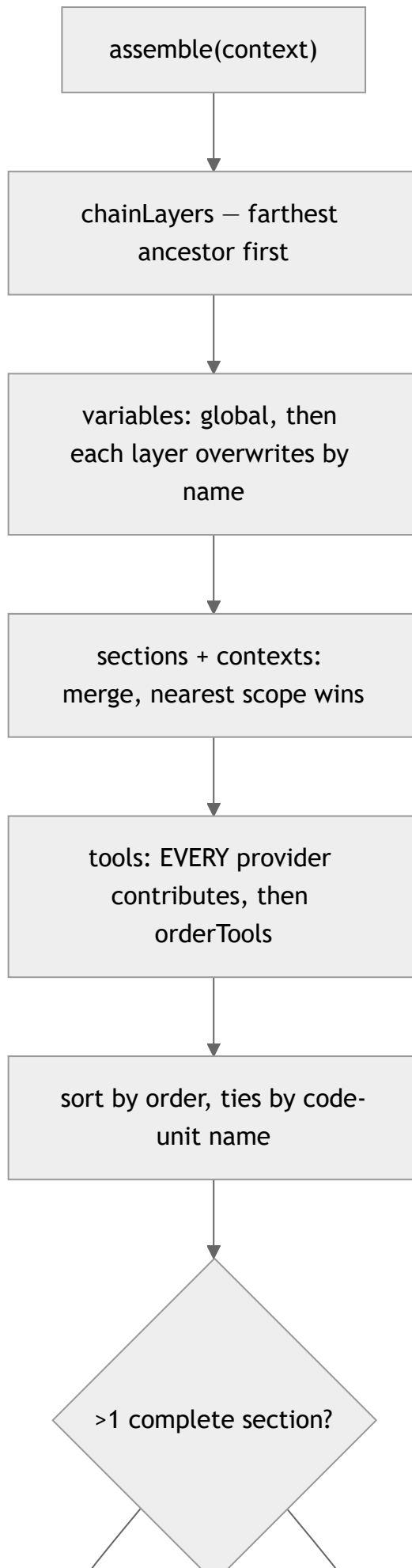
— system-prompt/src/index.ts:114–119

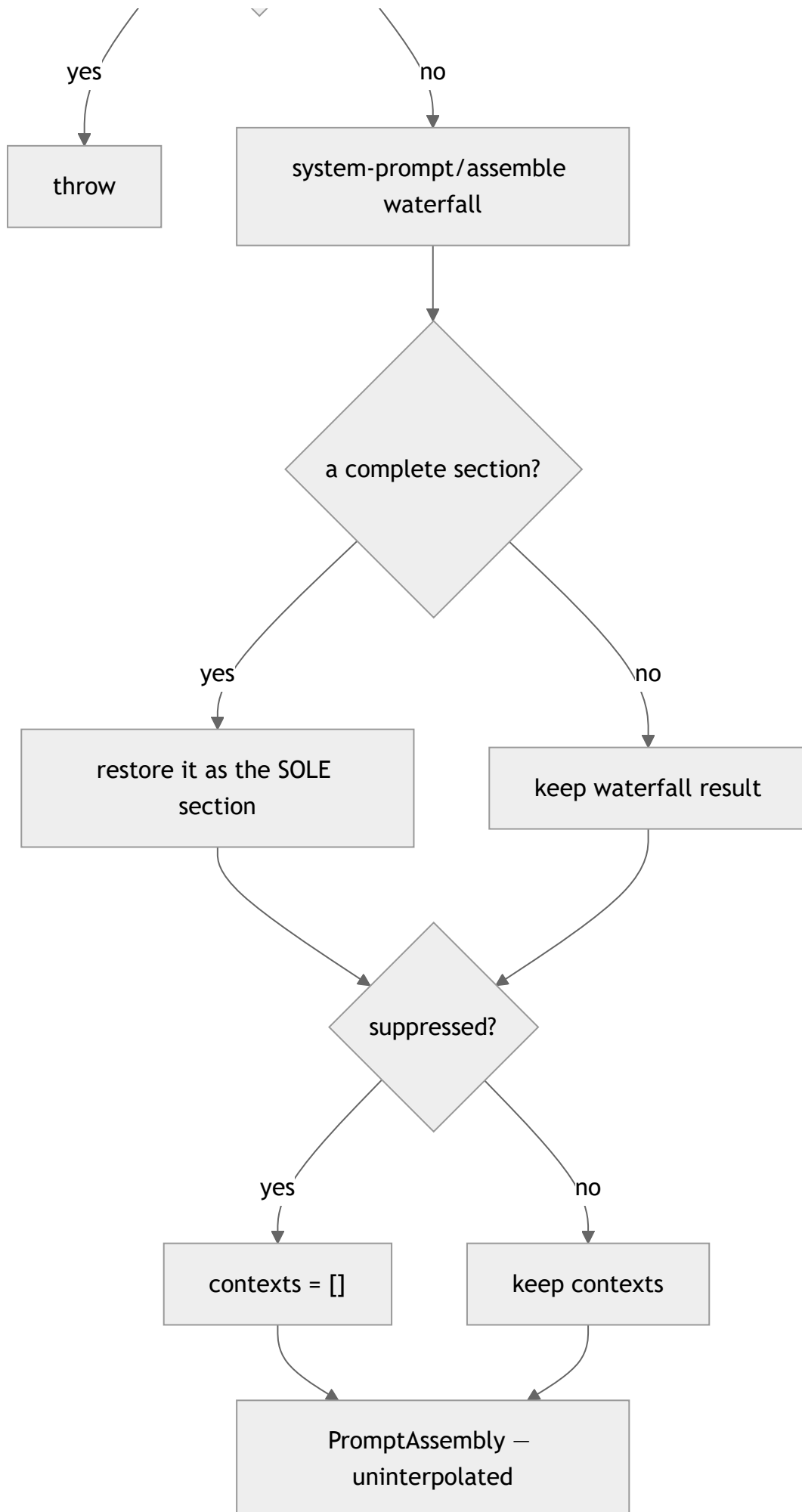
The sections/contexts split is about volatility. The system prompt is the provider's cache prefix; anything that changes mid-session goes in a context instead (§21). Text here is **uninterpolated**.

Registry	Collision behavior
section(s)	shadowed by name — nearest scope wins
context(c)	shadowed by name
tools(provider)	anonymous — all contribute
variable(name, fn)	overwritten by name

Tools accumulate rather than shadow because every mounted tool package contributes; uniqueness is enforced on tool *names* at ordering time.

Orders are centrally assigned — SECTION_ORDERS from HARNESS_IDENTITY: -1000 to STRUCTURED_OUTPUT: 9900, CONTEXT_ORDERS SANDBOX_POLICY: 110 / APPROVAL_POLICY: 115 / SUBAGENT_DELEGATION: 120 (:121–161). A test asserts every order is unique and ≥10 apart, leaving room to insert without renumbering. Ties break by **code-unit** name comparison (:222–224) — locale-independent, so the same contributions produce the same prompt on every machine, which prefix caching requires.





`assemble()` (:536-611) in order: chain layers farthest-ancestor-first → compute suppression → variables (nearest scope wins) → merge sections/contexts by **literal replacement by name** → concatenate all tool providers, then `orderTools` with a `TOOL_ORDER_REST` marker → sort by order → at most one `complete` section (more throws) → the `system-prompt/assemble` waterfall → restore a `complete` section as the sole section → force `contexts = []` if suppressed.

complete: true is live. The `minimal` preset uses it (`presets/minimal/agent.cordis.yml:13`) with `includeRuntimeContext: false`; `dsh-persona` passes both through (`preset/persona/src/index.ts:61`), the second calling `suppressRuntimeContext()` — **the producer of the suppression step above**. It makes an agent's prompt exactly one fixed string: identity, tool guidance, and any `assemble` listener are all discarded.

```
export function renderPrompt(assembly): string {
  return assembly.sections.map(s => interpolate(s, assembly.variables, 'section'))
    .filter(text => text.length > 0).join('\n\n')
}
```

— :263-268

Sections rendering empty **disappear** — conditional guidance without a conditional mechanism.

20 Variable interpolation

A hand-rolled single forward pass, not a global `replace`, and the reason is security: the cursor advances **past** each substituted value, so a `{{` inside a value is never re-scanned. A value containing `{{` — a code sample, a path, user text — cannot become an instruction.

```
for (let open = text.indexOf('{{'); open >= 0; open = text.indexOf('{{', last)) {
  const group = GROUP_AT.exec(text.slice(open))           // /^{\{([^\}]*)\}\}/
  if (group === null) {
    if (text.indexOf('}}', open + 2) >= 0) throw ...      // malformed
    result += text.slice(last, open + 2); last = open + 2; continue // lone "{{" is prose
  }
  const name = group[0].slice(2, -2)
  if (!VARIABLE_NAME.test(name)) throw ...
  if (!Object.hasOwn(variables, name)) throw ...          // never falls through Object.prototype
  const value = variables[name]
  if (value === undefined) throw ...                      // registered but valueless - distinct
  error
  result += text.slice(last, open) + value; last = open + group[0].length
}
```

— :309-346

Input	Result
<code>{{model}}</code> , registered with a value	substituted
<code>{{ model }}</code> , <code>{{a{b}}}</code> , <code>{{}}</code>	throws — malformed
<code>{{typo}}</code>	throws , listing every registered name
registered but <code>undefined</code>	throws , distinctly
use <code>{{</code> as an opener (no later <code>}}</code>)	literal prose

That last row surprises people: a lone `{{` is text, because prompt fragments legitimately discuss templating — but a `{{` that *does* have a later `}}` is almost certainly a botched reference, so it throws.

The engine registers exactly three variables — `provider`, `model`, `cwd` (`agent-loop/src/index.ts:414-416`), all reading from the agent and all able to return `undefined` for a bare `assemble()`. **Nothing interpolates at assembly time**, so a malformed template fails at the first render inside the loop, not at mount.

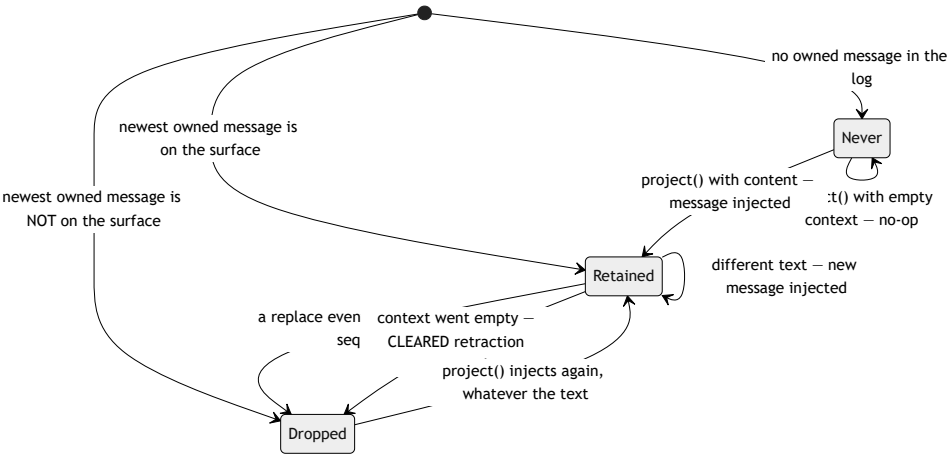
21 Runtime context injection

Volatile facts (sandbox mode, approval policy, delegation scope) arrive as a **message**, not in the system prompt, because a changing prefix invalidates the provider's cache for the rest of the conversation. But a message repeated every step accumulates — so it is injected **only when its text differs from the last still-visible one**.

```
private retained: { seq: number; text: string | undefined } | null | undefined
```

— `runtime-context.ts:27`

State	Meaning
<code>undefined</code>	no snapshot has ever existed in this session
<code>null</code>	one existed but is no longer retained — compacted away
<code>{seq, text}</code>	this exact durable event is the visible snapshot



State is **reconstructed from the log** on construction (`: 34-45`): walk events backwards for the newest `user/message` whose `source.plugin` is `@deepseek-ai/dsh-system-prompt`. A live listener keeps it current, and a **replacement whose `sourceEventSeqs` includes the retained seq clears it to `null`** (`: 46-55`) — §6's coverage rule is what makes that check complete rather than best-effort.

```
project(current, sections): UserMessage | undefined {
  if (this.retained === undefined && current.length === 0) return // nothing to say or retract
  const snapshot = current.length === 0 ? CLEARED : current
  if (this.retained?.text === snapshot) return // the dedup
  return createUserMessage({ content: [{ type: 'text', text: snapshot }],
    source: sections.length === 0 ? { kind: 'plugin', plugin: SOURCE }
      : { kind: 'plugin', plugin: SOURCE, form: 'snapshot', sections } })
}
```

— `: 64-75`

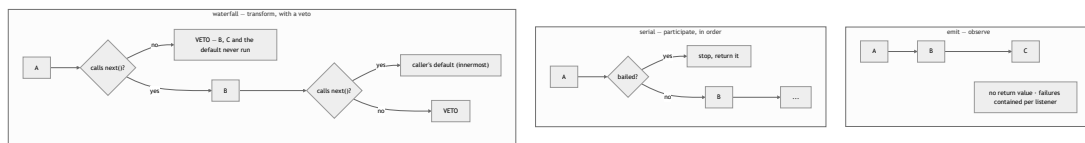
`CLEARED` is an explicit retraction — *"Current runtime context: none. Earlier runtime-context snapshots no longer apply."* Going quiet would leave the model holding a stale snapshot.

It enters as the **innermost default** of `agent/pre-step` (`agent.ts:239-249`): a listener that delegates inherits it; one that returns its own decision without calling `next()` **omits it**. `joinContextSections` supplies the fixed preamble — *"Current runtime context. This snapshot supersedes earlier runtime-context snapshots."* — because old snapshots remain in history and the model needs to know which is authoritative. Once appended it is an ordinary `user/message`, distinguishable only by `source.plugin`.

Only three producers exist repo-wide, matching the three declared orders: `sandbox:policy` (110), `approval:policy` (115), `subagent:delegation` (120).

⚠ **Naming trap:** none of `packages/context/**` — `agent-instructions`, `time-context`, `tmux-context`, `session-reference`, `file-reference-local` — uses `systemPrompt.context()`. They inject through `agent/pre-step` directly, and one registers an ordinary prompt *section*. "Context" there names a product category, not this mechanism.

22 Extension points



AgentEventDispatch (`agent/src/dispatch.ts:107-149`) is built once per agent and does two things per call: injects the `agent` into the payload (so a caller *cannot* dispatch about a different agent) and dispatches through the agent's scope carrier — *"so the scope key and the payload's agent cannot diverge."*

emit in raw Cordis has **no error isolation**: `this.dispatch('emit', args).map(cb => cb(...args))` (`events.ts:194-196`) — one throw aborts the rest. The agent dispatcher therefore resolves the list itself and wraps each call (`:120-137`). **Any code calling `ctx.emit()` directly inherits the hazard (§35)**.

waterfall (`events.ts:234-243`) is ten lines: listeners collected in registration order, the caller's default as the **innermost** link, each listener handed a `next` closure. A listener returning without calling `next()` **veto**es **everything after it, including the built-in default** — and there is **no framework enforcement**; `next` is an ordinary closure.

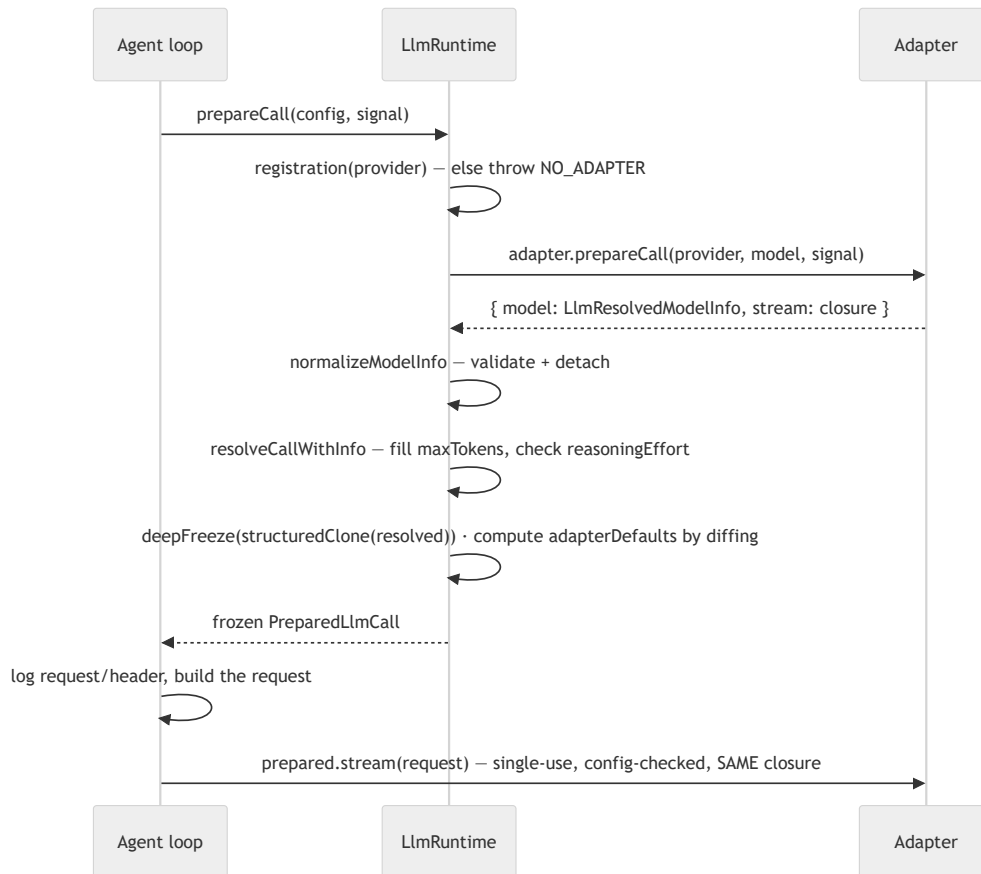
Live-listener census (shipped web profile)

Event	Mode	Live listeners
<code>agent/pre-step</code>	waterfall	12 — compaction, tool-skill ×2, checkpoint-policy, plan-mode, repeat-tool-reminder, goal-round-driver, agent-instructions, time-context, tmux-context, session-reference, subagent driver
<code>agent/request</code>	waterfall	1 — <code>installModelSelection</code> (<code>agent/src/model-selection.ts:54-70</code>), the <code>/model</code> command's implementation
<code>agent/request-error</code>	waterfall	2 — compaction, llm-retry
<code>agent/turn-stopping</code>	serial	0 — only the unmounted hook bridges implement it
<code>agent/status</code>	emit	3 — compaction (resets retry counters), session-controller, goal driver
<code>agent/created</code> / <code>agent/disposed</code>	emit	5 / 4
<code>agent/inbox/*</code>	emit	1-3 each
<code>agent/error</code>	emit	3
<code>agent/session-start</code>	emit	2

`agent/pre-step` is the dominant integration surface. `agent/turn-stopping` is defined, typed, and dispatched on every qualifying turn — **to nobody**.

23 · Adapters and `prepareCall`

Resolution and dispatch are **two stages** so the logged header and the sent request come from **one** resolution — between them the engine logs its header (§10), and configuration can change underneath.



Only `stream(options)` is abstract on `LlmAdapter` (:274); everything else defaults (:193-275).

`registerAdapter` (:380-409) runs inside `ctx.effect`, validates **all-or-nothing** (`DUPLICATE_ADAPTER`, `INVALID_ADAPTER` if `providerInfo.id` $\neq$ the registered name), and **captures the retry policy at registration time** (:429-430) — so changing it requires re-registering, which is why the handle exposes `.replace(providers)` for an atomic same-instance swap.

Selection is a **flat string lookup**; `model` is just a string handed to the adapter. The harness validates it against no catalog — `listModels` is advisory, and DeepSeek treats an unrecognized model as text-only rather than rejecting it (`llm-deepseek/adapter.ts:398-407`).

`prepareCall` (:890-935): find registration $\rightarrow$ adapter prepares $\rightarrow$ `normalizeModelInfo` validates (provider/id match, non-empty name, positive-integer `contextWindow`, safe-integer `defaultMaxTokens`, well-formed `reasoning.efforts`) $\rightarrow$ `resolveCallWithInfo` fills `maxTokens` from `defaultMaxTokens` and resolves effort (`UNSUPPORTED_REASONING_EFFORT` if the model declares no reasoning, or the effective effort is not in `efforts`) $\rightarrow$ freeze $\rightarrow$ **compute adapterDefaults by diffing caller vs resolved** (:899-906).

The returned `stream` enforces three things (:917-932): **single use** (`INVALID_PREPARED_CALL` on reuse — so a retry cannot reuse a stale resolution, which is why §9's retry rebuilds from scratch); **config equality** with what was resolved (so the logged header cannot lie); and dispatch through the **same adapter closure** captured at prepare

time (so a settings change or hot reload cannot mix generations). Concrete adapters lean on this — DeepSeek snapshots its whole connection once per call (`adapter.ts:432-438`).

Decision	Condition	Site
NO_ADAPTER	provider not registered	:937-941
DUPLICATE_ADAPTER	name held by another registration	:422
INVALID_ADAPTER	<code>providerInfo.id</code> $\neq$ the registered name	:388, 420, 426
INVALID_MODEL_*	model metadata fails validation	:747-808
UNSUPPORTED_REASONING_EFFORT	effort on a non-reasoning model, or not in <code>efforts</code>	:859, 868
INVALID_PREPARED_CALL	reuse, or a config mismatch at dispatch	:918, 921
Set <code>adapterDefaults.X</code>	caller omitted X and the resolved config has it	:899-906

`forAdapter()` (:944-957) strips `replayState` from historical assistant messages whenever the current target's adapter instance differs from the one that produced them: opaque provider state is trusted only back to the same live adapter.

24 Streaming and block assembly

`BlockAssembler` (`llm/src/assembler.ts`) keys partials **by stream block index**, not arrival order — blocks interleave. `block-end` sets `.block` on **first** close and is **authoritative**; later deltas for that index are ignored as stragglers (:65-72). An open block of an unknown type at assembly **throws** rather than being guessed at (:119).

The max-tokens rule (:134-150): when `finish.kind === 'max-tokens'`, **every tool-call block is dropped**. A truncated call's arguments are incomplete JSON; executing it would act on a request the model never finished making. Text and reasoning survive. `replayState.blocks` is pruned to match, or discarded if lengths diverge.

Two consequences: §9 returns `{kind: 'max-tokens'}` and that ending is **sticky**; and the resulting assistant message may have **zero content blocks** — still logged to carry usage, and §7 skips it so it never enters the transcript.

`interruptedBlocks()` (:169-179) returns only text/reasoning with non-whitespace content, omitting tool calls because *"interruption precedes dispatch; retaining one would require a fabricated result."* The engine calls it **only** when the signal actually aborted, appending `interrupted: true` with the collected chunk seqs, then rethrows.

`finish` defaults to `{kind: 'stop'}` when no finish chunk ever arrived (:187-189) — a provider closing after its last block is a clean completion. **Every chunk is logged before assembly**, so the raw stream outlives the interpretation — which is why §30's packing codec exists.

✅ `assembler.message()`'s default `source` is **verified dead** — no non-test caller anywhere in `packages/llm`, `packages/core`, or `packages/compaction`.

25 Failures and retry

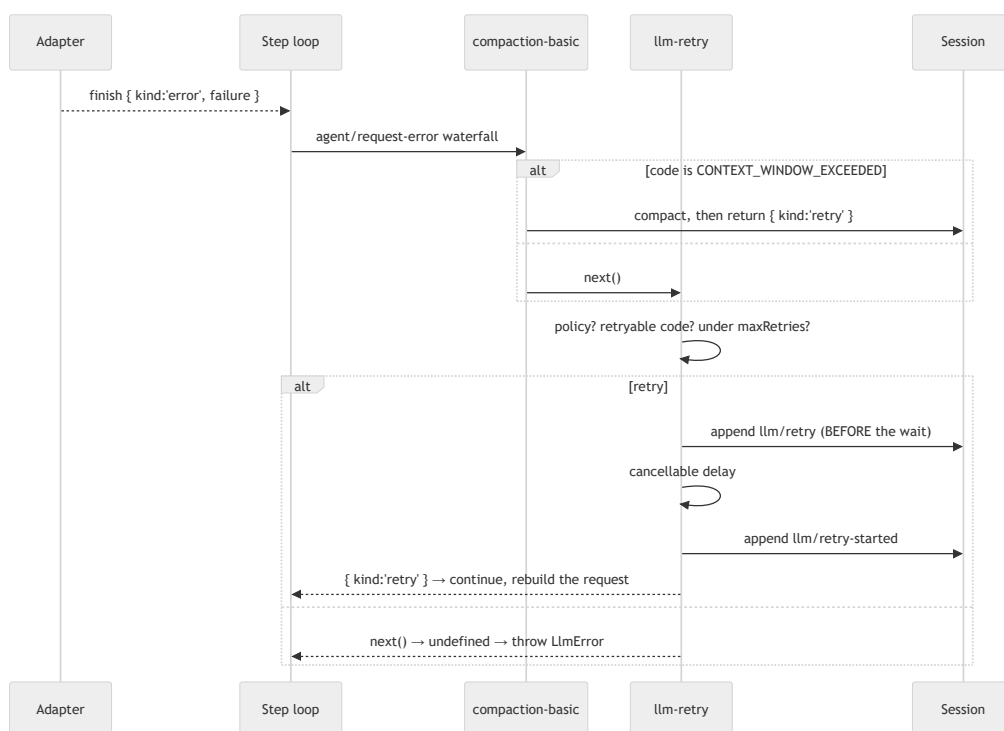
Two separations: **description from handling**, and **policy from mechanism**.

`normalizeLlmFailure` (`llm/src/adapter-failure.ts`) is the single point converting any thrown value into a serializable `LlmFailure` `{message, code, status?, providerRetryAfterMs?, requestId?}`. It is defensive: reads `code` / failure **only via** `Object.getOwnPropertyDescriptor`, never property access, so a getter cannot throw or lie during error handling; trusts a carried snapshot only when its `code` agrees; and assigns `UNKNOWN` to anything non- `Error` or foreign — *"Trust only Harness-owned codes; third-party SDK codes are not our taxonomy."*

Group	Codes
Registry	NO_ADAPTER, DUPLICATE_ADAPTER, INVALID_ADAPTER, INVALID_PREPARED_CALL, INVALID_MODEL_*, UNSUPPORTED_REASONING_EFFORT
Credentials	INVALID_CREDENTIAL, MISSING_CREDENTIAL
Transport/provider	AUTH, INVALID_REQUEST, RATE_LIMIT, SERVER, HTTP_<status>, QUOTA, CONTEXT_WINDOW_EXCEEDED
Adapter-local	TRANSPORT, ABORTED, TIMEOUT, MALFORMED_RESPONSE, STREAM_CLOSED, UNSUPPORTED_CONTENT, EMPTY_RESPONSE

⚠ Provider errors are classified by **regex-matching the provider's error text** (`error.ts:80-100`) — the most fragile behavior-critical coupling in the system (§35).

Policy is pure data (`retry-policy.ts:14-24`): `maxRetries = 5`, `initialDelayMs = 500`, `maxDelayMs = 10_000`, `jitterRatio = 0.1`, `retryableCodes = [EMPTY_RESPONSE, RATE_LIMIT, SERVER, TIMEOUT, TRANSPORT]`. Note what is **absent**: `AUTH`, `INVALID_REQUEST`, `QUOTA`, `CONTEXT_WINDOW_EXCEEDED` — retrying those unchanged would fail identically. Overflow is retryable only *after something changes the history*, which is why compaction owns that path.



The engine contributes the seam and a terminal default. `RequestErrorAction` is `{kind:'retry'}` | `undefined`, and `undefined` "leaves the failure terminal" (`runtime-types.ts:255-256`).

`llm-retry` (:194-258, mounted `base:84-85`) supplies retries:

- **policy === undefined** → **delegate immediately** (:198). A request whose `preparedCall` was `undefined` — the `NO_ADAPTER` fallback — is **never retried**. That is exactly the fresh-install case (§32).
- normal gates on `retryableCodes` and enforces `maxRetries`; **always has neither gate nor cap** and retries indefinitely until abort or disposal (§35).
- **Retry counting is durable**: `ctx.sessionProjections.stateOf(agent.session, 'llmRetry')` (:220), folded from `llm/retry` events keyed by `[provider, policyKey]`, cleared on `step/start / turn/end`. A budget survives a process restart because it was never in memory.

- Ordering: append `llm/retry` → **cancellable wait** → append `llm/retry-started` (:149–192). "Each scheduled retry is durable before its cancellable wait" — a crash during backoff leaves evidence.
- `providerRetryAfterMs` is honored when present and within `maxDelayMs` ; otherwise bounded exponential backoff with symmetric jitter.

26 Measuring and spilling

Two separate questions, different mechanisms, different times.

The token meter prefers real provider usage, but **only when it is at least as large** as a full heuristic re-price of the same anchor (`token-meter/src/index.ts:154–156`) — a smaller reported usage would under-report and compact too late. Otherwise it falls back to a fixed density: `CHARS_PER_TOKEN = 4` , `BLOCK_OVERHEAD = 4` , `ROLE_OVERHEAD = 4` (`estimate.ts:13–19`).

This is not a tokenizer — no BPE table, nothing provider-specific. The design compensates by preferring real usage as soon as one request succeeds, and by setting the threshold at 80% rather than 100% (§35).



Spill acts at *tool-execution time*, before an oversized result ever becomes a message. Three packages: `spill` defines the seam (`saveText(input): Promise<SpillRef>` , deliberately no retention policy or retrieval API); `spill-local` writes to a private session-scoped path (`0700` dir, `0600` file) and returns a locator plus the hint "Use read with offset/limit, or grep this path to search within it" — addressed to the model, reusing tools it already has rather than inventing a retrieval tool; `spill-policy` decides, as a **prepended** `tools/post-execute` listener.

Algorithm: no-op unless `maxInlineBytes` configured → **plain text only** (spilling an image would break it) → under the cap untouched → over it, save and replace with a bounded head/tail preview plus a notice → **best-effort**, a save failure logs and keeps the original inline.

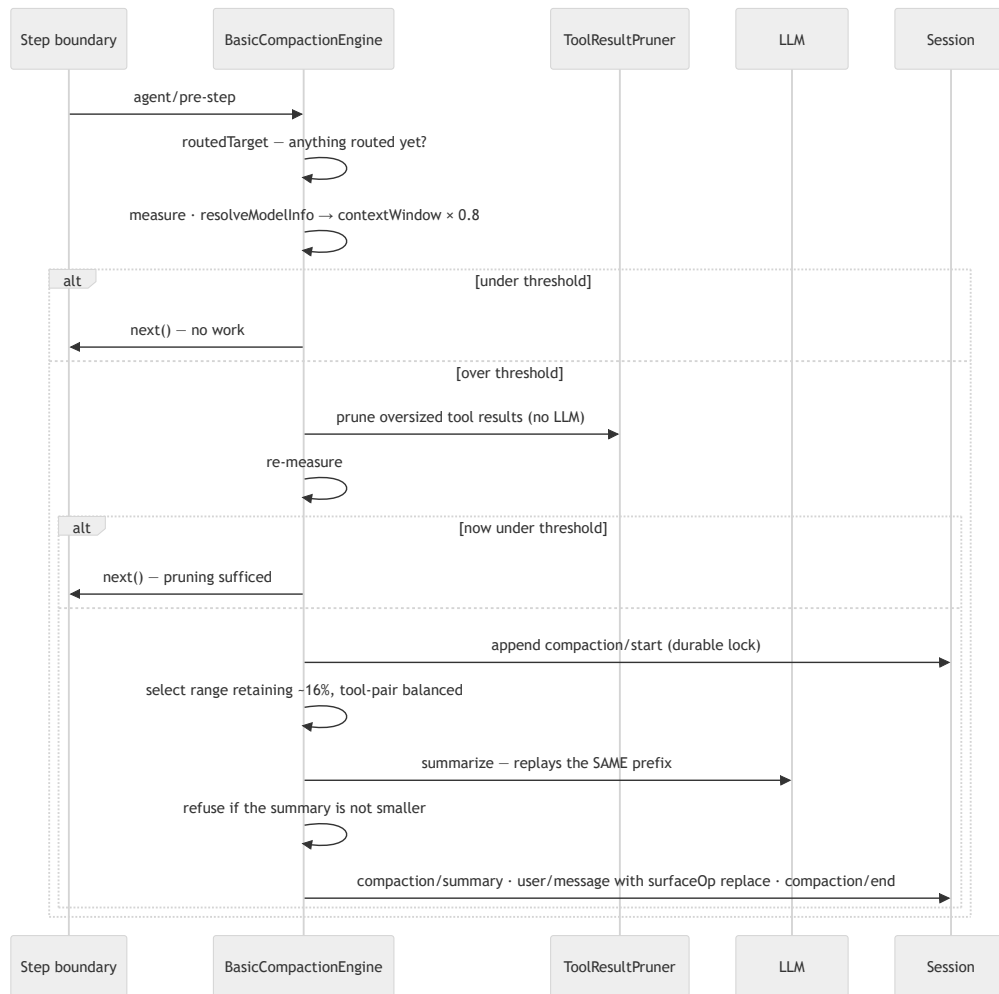
The `read` tool is **excluded** (:197) — otherwise `read` → `spill` → "use read on this path" → `read` → `spill`, forever.

🧹 The sweep **does not follow symlinks** — its own tests assert "does NOT follow a symlinked session directory" and "excluding symlinks and non-matches." A cleanup deleting *through* a symlink would let anyone able to plant one direct deletions anywhere. (Those two cases are the only failures when running this suite on Windows — not because the property is broken, but because Windows refuses to *create* the test's symlink: `EPERM`.)

! **Confirmed limitation:** the sweep deletes by `mtime` alone (:40–46) and knows nothing about which sessions are live. A session resumed after `cleanupPeriodDays` (default 30) holds `spill` notices pointing at deleted paths; the model gets a file-not-found. `cleanupPeriodDays: 0` disables cleanup. The spilled text was never part of the durable record, and the logged preview survives.

27 Pruning and compaction

Cheapest first: **pruning** is model-free head/tail truncation of individual oversized tool results; **compaction** LLM-summarizes a *region* and replaces it. Both use §6's `replace` . Neither deletes anything.



Two triggers (`compaction-basic/src/index.ts:138–225` , gated by `auto` , default `true`):

- **Pressure** — an `agent/pre-step` listener that **always calls** `next()` (`:165`); it never rejects a step, only mutates history as a side effect. Checking every step means deciding with the exact history the next request will use.
- **Overflow** — an `agent/request-error` listener firing only on `CONTEXT_WINDOW_EXCEEDED` (`:180–224`). The threshold check is **skipped entirely**: the provider already said the window was exceeded; arguing via a character estimate would be absurd. It returns `{kind: 'retry'}` **only if** `replaceGeneration` **actually advanced** (`:219–223`) — an honest claim — otherwise it delegates. Capped by `maxOverflowRetries` (default `1`), tracked per agent and reset on the next `assistant/message` or on idle.

Range selection (`region.ts:100–136`): walk the surface backwards accumulating until `retainTokens` ($\approx 16\%$ of the window) is preserved verbatim, then forward to the nearest boundary that does **not split a tool-call/result pair**. The most recent conversation always survives; the boundary never orphans a call from its result.

compaction/start is the lock (`:191`), written synchronously before any async work;

`assertCompactionInactive` rejects a second attempt and re-checks after every `await` . It is a log event, not a mutex — so it survives a crash and is visible to a reader. On failure the code still appends `compaction/end` carrying an `error` (`:224–230`): the lock is always released.

Summarization (`summarizer.ts:121–182`) replays *the conversation's own last routed system prompt, tools, and shadowed messages verbatim* — reusing the exact prefix keeps the provider's KV cache warm — then appends one fixed instruction asking for a structured checkpoint (Primary Request and Intent · Key Technical Concepts · Files and Code · Errors and Fixes · Pending Jobs · Current Work · Next Step · Critical Context) and calls `ctx.llm.stream` with `purpose: 'compaction'` . This is the one-shot call §11 deliberately excludes.

A compaction that would not shrink is refused (:383-388) — otherwise a terse session near the threshold would compact repeatedly, spending a model call each time.

Commit (:437-488): `compaction/summary` (text, provenance, shadowed range and seqs, token count) → a user/message framed in `<compacted-summary>` tags with `surfaceOp: {op: 'replace', start, end}` and `sourceEventSeqs: [startEvent.seq, summaryEvent.seq, ...shadowedSeqs]` → `compaction/end`. The provenance cites **more than the coverage rule requires**, so the whole transaction is reconstructable, not just its effect.

Manual /compact (:369-421) differs three ways: requires idle via `agent.runMaintenance` (throwing `ManualCompactionError('busy')`), selects with `retainTokens: 0`, and uses `stability: 'selected-span'` rather than `'whole-surface'`.

Downstream effects of one compaction: the derived cache rebuilds (§7), the next header logs `series` despite identical bytes (§10), a shadowed runtime-context snapshot re-injects (§21), and the human transcript is **unaffected** (§6).

Mount status took three layers to establish: base-mounted, web-app-disabled, re-mounted by the standard preset inside `isolate: {compaction, toolResultPruner}` (preset:137-155). Live per agent. A two-layer reading concludes compaction is off.

28 Subagents

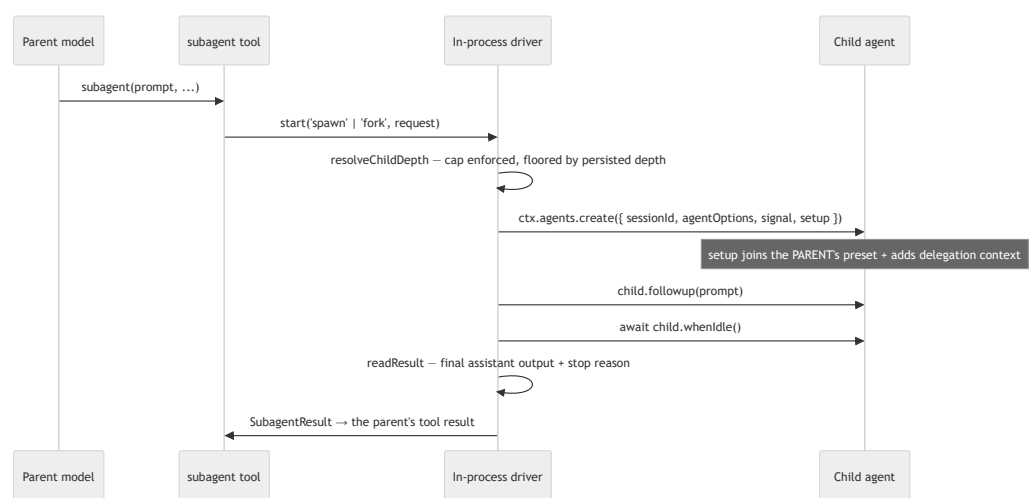
A subagent is a **genuine nested Agent** built by the same factory — there is no lightweight path:

```
const handle = await parent.ctx.agents.create({ sessionId: childId, meta, agentOptions, signal, setup
})
```

— `subagent-in-process-driver/src/index.ts:133-140`

It has its own session, log, inbox, compaction, and can itself delegate.

Provider	Child's session starts	For
spawn	<code>empty (inheritsParentContext = false)</code>	independent work
fork	seeded with the parent's completed turns	context reuse; the shared prefix stays KV-cache eligible



Depth cap defaults to 3 — and it is the *tool's* config, not the runtime's:

```
maxDepth: z.union([z.natural().max(Number.MAX_SAFE_INTEGER), z.const('provider-managed')]).default(3)
```

— `tool-subagent/src/index.ts:129`

`resolveChildDepth` treats the cap as optional; with nothing supplying one there is no limit beyond safe-integer range. The tool refuses to mount a numeric cap against a provider lacking the `depthLimit` capability (:323-326). The **persisted floor** is what makes it robust:

```
return Math.max(agent.session.header.delegationDepth ?? 0, runtime ?? 0)
```

— `depth.ts:36` — *"runtime may DEEPEN the count but can never lower it — a resumed child arrives with fresh options, and counting it from zero would let it delegate as if it were top-level."*

Composition: `applyChildComposition` calls `childCtx.get('agentPresets')?.composeFrom(childCtx, parent.ctx)` (`child-agent.ts:204`) — the child **joins the parent's mounted preset** rather than getting an empty plane — and injects a fixed `SUBAGENT_DELEGATION_CONTEXT` telling the model its scope is fixed and cannot be widened.

Results come back *from the child's own log*: `child.followup(prompt) → await child.whenIdle() → extract` the final assistant output after the activation boundary and map the turn-end reason to `completed | max-tokens | aborted | refusal | error`. Continuable children instead push through `reportFrom()` at any time, not only at turn end.

Planes: the `subagents` registry and its spawn/fork backends stay **host-plane** (a process singleton whose cross-session queries the API serves; a provider name may register only once). `tool-subagent-report` is host-plane too, for a sharper reason — it registers a *continuable setup* on that singleton, and "the setup list is not scope-aware — one copy per mounted preset means every child gets `report` registered once per live session, which throws on the second." A preset chooses only which delegation **tools** its agent sees.

Note the base declares `tool-subagent-fork` as `one-shot`; the preset re-declares it **continuable**, and that is what runs. The preset's own comment accepts the tradeoff: a continuable fork's `report` tool and prompt section precede the inherited history and invalidate the very prefix forking exists to preserve.

29 Persistence

Two layers: a **backend** knows bytes (`loadStored`, `readStoredRevision`, `appendBatch`, `commitRepair`, `list`, plus optional `seek/materialize/locate/close` — `coordinator.ts:128-219`); the shared **PersistenceCoordinator** knows correctness (batching, per-id serialization, crash-repair sequencing, an LRU of unpublished prepared sessions, dispose-time draining) and is composed by every first-party backend.

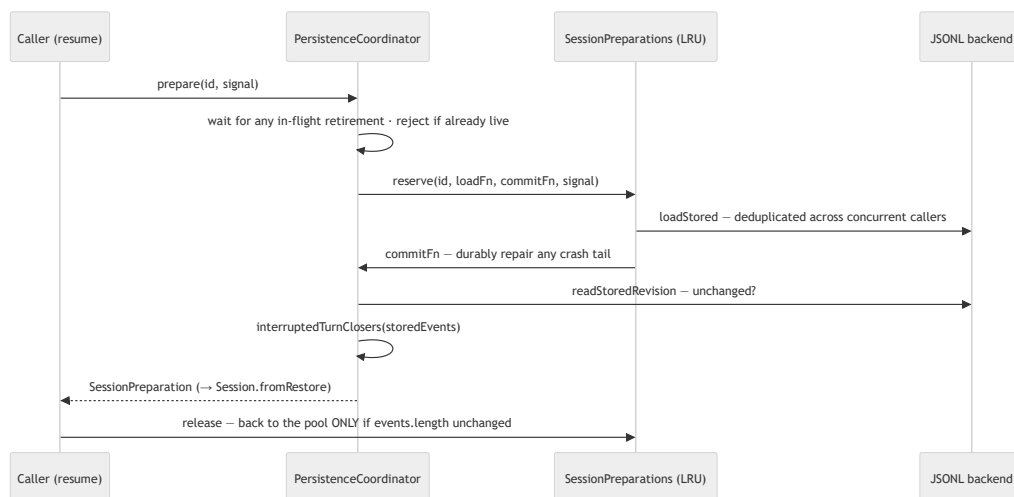
What runs: JSONL, one append-only file per session — header line plus one line per event, in project directories, with Zstandard compression and chunk packing both on by default (`base:110-113`).

SQLite exists but is not the log: `session-query-sqlite` is an FTS5 *search index*, mounted `path: ':memory:'`, `openAt: never` — `ctx.sessionQuery` stays available for exact reads, titles, and lineage traces while **SQLite is never opened** and content search fails `SESSION_QUERY_SEARCH_DISABLED`.

Writing safely. Materialization uses `link()` + `unlink()`, **not** `rename()` (`jsonl:560-565`): `rename` silently replaces, `link` fails `EEXIST`, so a two-process race is detectable instead of destructive. Appends are `fsync`'d with **truncate-back-to-pre-write-size** rollback (:670-698). Batch seq contiguity is re-checked before writing (`coordinator.ts:722-726`).

Reading safely. Two gates, both loud: `assertVersion` (directional messages for newer vs older) and `assertEventsSupported` (§5).

Decision	Condition	Site
Refuse the session	header version $\neq$ <code>SESSION_FORMAT_VERSION</code>	:1128–1131
Refuse the session	unknown required event type	:1143–1148
Refuse the write	batch seq $\neq$ <code>cursor + i</code>	:722–726
Roll back	append failed mid-write	jsonl :670–698
Fail materialization	destination exists (<code>EEXIST</code>)	jsonl :560–565
Reject preparation	the id is already live	:744–771
Reuse a reservation	the caller did not mutate it	:764–767
Adopt rather than repair	the live prefix matches disk	:1383–1405



SessionPreparation (`core/session/src/preparation.ts:20–49`) is a `Disposable` wrapper around one *unpublished* session, so a caller can build it, decide whether to publish, and cleanly release backend state if not. `coordinator.prepare` (:744–771) waits for any in-flight retirement of the id, **rejects if the id is already live**, reserves through a structure that de-duplicates concurrent cold reads, and returns a preparation whose `release` returns it to the pool **only if untouched**:

```
reservation.source.session.events.length === reservation.source.sessionLength
```

— :764–767

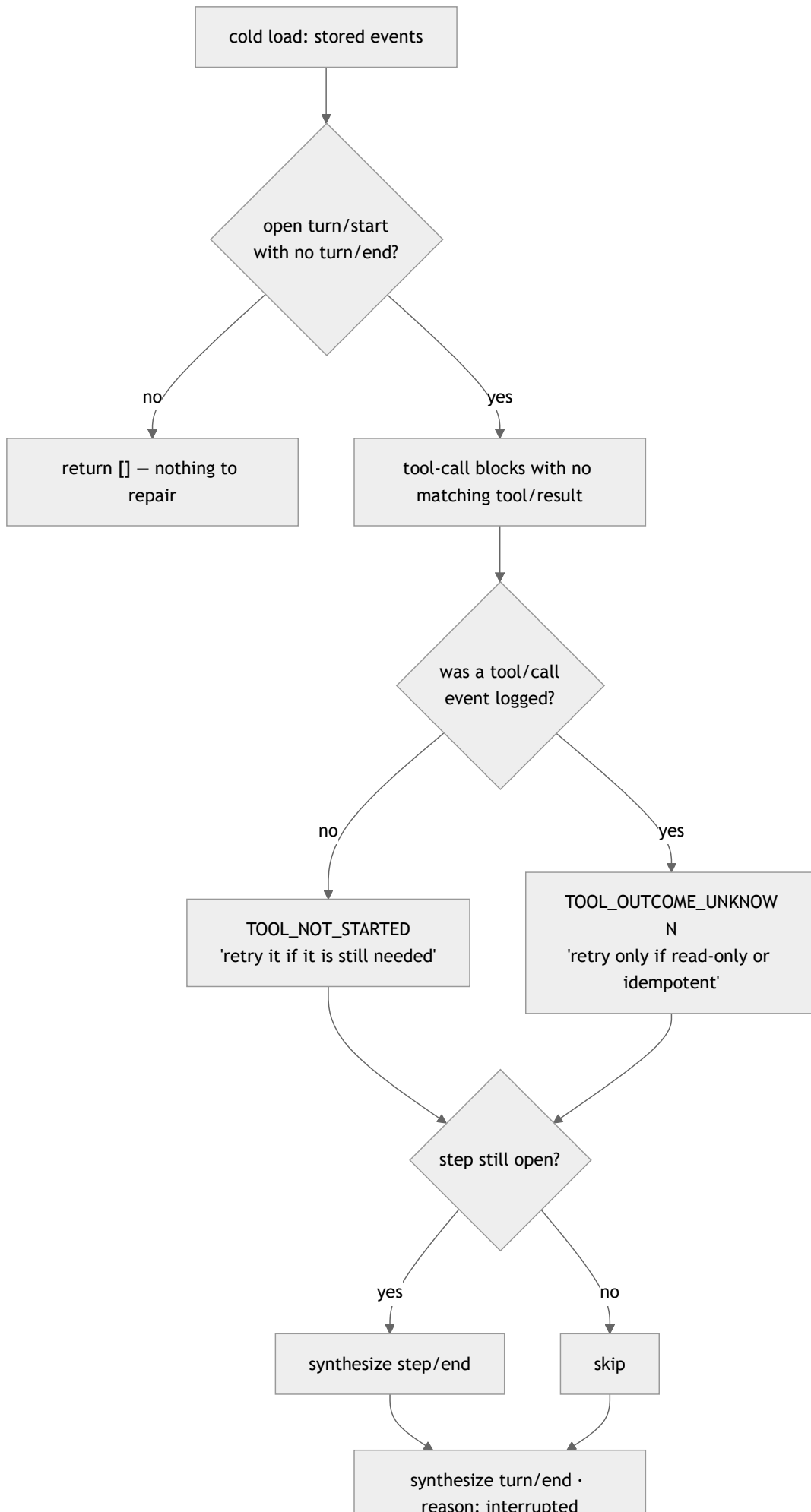
`prepareCore` (:974–1013) loads the stored prefix, upgrades legacy shapes, computes crash-tail closers (§30), appends them to the seed, and constructs via `Session.fromRestore`.

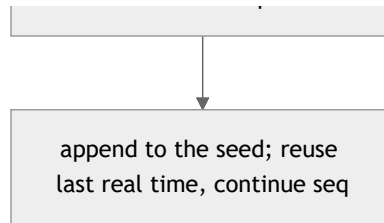
Adopting a live session (`onCreated` , :1318–1375) handles four cases — already tracked, matching on-disk prefix, mismatched artifact (reject as collision), genuinely new. The matching case routes through `adoptLivePrefix` and explicitly **does not** use cold `prepareCore`, because that would crash-repair a turn the live session is still extending.

`SessionStore.fork()` copies a prefix and **rejects a boundary inside an open turn** (`SessionForkError`, code `OPEN_TURN`) — same reasoning as compaction's tool-pair balancing.

30 Crash repair and chunk packing

Repair





`interruptedTurnClosers` (`core/session/src/repair.ts:28-134`) is pure and returns `[]` for a balanced log. The branch at **E** is decided purely by whether a `tool/call` event exists — which is exactly why §17 appends it *before* `prepare` runs.

That two-code distinction is the most careful prose in the engine: the system genuinely does not know whether the call took effect, so rather than choosing for the model it hands over the one fact that matters and the criterion to reason with — *"retry only if the operation is read-only or idempotent... Do not retry blindly."*

`turn/end {kind: 'interrupted'}` is written **only** here — the loop never emits it. Synthetic events reuse the last real event's `time` ("never invents a 'future' time") and continue its `seq`. Repair runs **only on the cold path**; a live session being adopted skips it. Byte-level damage (a half-written line) is a separate concern handled by the backend's torn-marker mechanism.

Packing

"Packed rows are an encoding vocabulary, NOT session events: they never enter `Session.events`, have no `SessionEventMap` entry, and use bare (slash-less) type tags so a reader cannot confuse them with the event taxonomy." — `chunk-rows.ts:9-12`

Three row tags — `text-chunks`, `reasoning-chunks`, `tool-call-chunks` — carrying `seq0`, `time0`, and `dt` (epoch-**ms gaps**, one fewer than the members; member k is `seq seq0 + k` and `time time0` plus the first k gaps). Gaps may be negative: the wall clock can step backwards, and the format says so rather than assuming. Text runs keep `texts[]` **never joined** — token boundaries are data. `MIN_RUN = 3`, documented as a **format constant, not a tunable**, because both layouts decode identically.

Round-trip safety is where the module spends its code: `classify` whitelists **exact key sets and primitive types**, anything unrecognized storing verbatim ("*unknown fields or future chunk variants lose compression, never data*"); `continues` refuses to extend a run when `next.time - prev.time` is not a safe integer (a rounded gap would decode to a different timestamp); `validateRow` re-checks that every reconstructed `seq` and `time` stays in safe range, and **throws** on a malformed row because "treating it as an event would silently drop a whole run."

⚠ The module claims a **~56×** envelope-overhead ratio "measured on a real DeepSeek session" — quoted as the authors' figure, not verified here.

Real example (`text-turn/session.jsonl:15`): twenty reasoning deltas in one line, nineteen gaps all zero but one. Decoded, twenty `assistant/chunk` events. The model's tokenizer split "PONG" across " \", "P", "ONG", "\" — which joining would lose.

Part IV • The engine in motion

31 One real turn, annotated

snapshots/session/text-turn/ — prompt: *"Reply with exactly the word: PONG. Do not use any tools."* The smallest complete turn, so every event present is one the engine cannot skip.

Line numbers are not sequence numbers. The file has 25 lines but the session has **43 events**, because line 15 is a packed chunk row holding twenty. The mapping is verifiable from the data itself, by two citations produced by different mechanisms at different times: `session/title` carries `messageSeqs: [7]`, and `assistant/message` carries `sourceEventSeqs` spanning **12–39** (28 values, matching $39 - 12 + 1$). Both agree with one arithmetic model — §6's provenance system verifying a reading of itself.

Line	Seq	Event	Mechanism
1	—	session header — version:0, delegationDepth:0	not an event ; storage metadata (§5)
2–4	0–2	permission/preset, sandbox/mode, approval/policy	three plugins the engine knows nothing about; none surface-eligible
5	3	agent/inbox/spliced target next-turn	followup() ; the queue is the log (§12)
6	4	turn/start turn 1	turn 1 because turnBoundary folded an empty log (§8)
7	5	agent/inbox/spliced removedCount:1	claim() popping one ; logged <i>before</i> step/start
8	6	step/start	
9	7	user/message, source.kind: 'user'	the human's prompt, now on the surface
10	8	user/message, source.plugin: dsh-system-prompt, form: 'snapshot'	the runtime-context snapshot (§21)
11	9	session/title, source.kind: 'fallback', messageSeqs: [7]	a plugin; same provenance discipline
12	10	request/header, reason: 'initial'	first request of this instance over a log with no header (§10)
13	11	request/context	
14	12	assistant/chunk block-start idx 0 reasoning	
15	13–32	20 packed reasoning-delta	one storage row (§30)
16	33	block-start idx 1 text	blocks interleave — 1 opens before 0 closes
17–18	34–35	text-delta "P", "ONG"	
19–20	36–37	block-end idx 0, idx 1	authoritative closes (§24)
21	38	usage — 3091 in / 23 out / 20 reasoning	
22	39	finish {kind: 'stop'}	
23	40	assistant/message, sourceEventSeqs: [12...39]	cites all 28 chunk events
24–25	41–42	step/end, turn/end {kind: 'completed'}	no tool calls → completed (§9 ⑨)

Line 10 is the single best illustration in the book. That message: was produced because `retained === undefined` and context was non-empty; entered as the **innermost default** of `agent/pre-step`, so every listener inherited it by delegating; carries the two sections lines 3–4 caused, each attributed to its plugin; and was appended as an ordinary `user/message` — **indistinguishable from line 9 except by its source tag**. Its second section reads *"do not request sandbox escalation (do not set `sandbox_permissions`)"* — the policy from line 4 enforced and explained, so the model does not waste a turn on something auto-rejected (§18).

The cost: 43 events, 2 surface nodes, 2 messages sent, 1 model call, **3,091 input tokens for a 23-token answer** — almost all of it system prompt and 35.5 KB of tool schemas, for a request told not to use tools. That is the standing price of a stable catalog, and the reason it *stays* stable (§15).

32 When things go wrong

A • Context overflow — recovers

Adapter throws → `normalizeLlmFailure` recognizes the provider's text → `CONTEXT_WINDOW_EXCEEDED` → terminal error finish → `agent/request-error` waterfall → **compaction claims it**, skipping the threshold entirely (the provider already answered) → prune first, re-measure → if needed, lock, select a tool-pair-balanced range retaining ~16%, summarize replaying the same prefix, refuse if not smaller, commit as `replace` → `return {kind: 'retry'}` **only if `replaceGeneration` advanced** → `continue` rebuilds the request.

Four things differ in the rebuilt request: the derived cache rebuilt (§7); the header logs `series` despite identical bytes (§10); the runtime-context snapshot may re-inject (§21); the prepared call is new, the old one being single-use (§23). Capped at `maxOverflowRetries` (default 1).

B • Cancellation mid-stream — unwinds cleanly

`cancel(cause)` clears the inbox and the latch, then aborts. Partial output is salvaged via `interruptedBlocks()` and logged `interrupted: true`, then the error rethrows. **In-flight tools drain** and commit in model order; **unstarted calls get synthetic `ABORTED_BEFORE_DISPATCH` results** so replay stays valid. The turn records `{kind: 'aborted', reason}` — the cause, not just the fact. `step/end` and `turn/end` come from `finally` blocks, so the log stays balanced. A message arriving in the aborting window is retargeted to `next-turn`, classified *before* the splice.

The property preserved: a cancelled session is resumable — balanced boundaries, every call paired with a result, partial output kept and marked. **Where it can still stall:** convergence is cooperative with no watchdog.

C • No provider configured — cannot be fixed by the engine

The most surprising path, and it took three composition layers plus two plugins to establish.

`llm-deepseek` is `disabled: true` in `web` (`web-app:41-42`) — *"The Web surface exposes only providers declared through the Models settings document."* `llm-pi-ai` **is** mounted but its `providers` dict defaults to `{}`, so `ensureRegistrationFacts` sees `routes.length === 0`, records the facts and **returns without registering anything** (:284–287) — the documented "dormant posture."

So: **zero routes registered**. `prepareCall` throws `NO_ADAPTER` → `buildRequest` catches *only that code* and proceeds unresolved, `preparedCall` stays `undefined` → the fallback `llm.stream` throws `NO_ADAPTER` again → terminal error chunk → compaction delegates (wrong code) → **llm-retry receives `retryPolicy: undefined` and its first line is `if (policy === undefined) return next()`** → nothing else listens → the turn fails.

The fix is a settings write, not re-enabling the row. `llm-pi-ai` installs a settings section whose `onChange` re-runs `ensureRegistrationFacts`, which on the first non-empty profile set calls `registerAdapter` (:288); later changes use `registration.replace(routes)`, an atomic same-instance swap. No restart, no composition change.

And `registeredFacts` only advances once the registry actually holds the new set, so a refused update **keeps the previous routes serving**.

	A • Overflow	B • Cancellation	C • No provider
Recoverable	automatically	n/a — user-initiated	only by configuration
Handled by	a plugin on an extension point	the loop's own <code>finally</code> blocks	nobody
Log stays valid	yes	yes	yes
Turn ending	<code>completed</code> after retry	<code>aborted</code> with cause	<code>error</code> with a code

All three leave a **resumable session** — the design's strongest claim, each path testing a different part of it.

Part V • Extending and changing

33 Adding a tool

Six decisions determine whether a tool behaves correctly under replay, cancellation, concurrency, and compaction.

Decision	Getting it wrong costs
What is the canonical value ?	replay cannot re-render; the UI has nothing structured
Concurrency-safe ?	data races, or needless serialization
Needs approval ?	a security hole, or an unusable tool
Host plane or preset ?	a service collision, or an invisible registry
What does it present ?	a wall of text where a diff belongs
Should it conclude the turn ?	almost always no

Value, not prose. `bash` returns `{kind:'foreground', exitCode, signal, timedOut, ...}` so a UI can parse the exit code into a pill and replay re-renders without re-running; `str_replace_editor` returns a bare string and gives that up. Rule: if any consumer might branch on part of the result, it belongs in the value. And **value is never persisted** — use `meta` for replay-time data.

Placement, from the preset's own comments: a row that **publishes a service** must sit in an `isolate` realm or it becomes process-global and collides; a **registry something outside the preset reads** belongs to the host plane. Worked cases — `tools / skill / jobs / subagents` registries and `token-meter` are host-plane; `tool-bash / tool-fs / tool-web` are preset rows that register *into* the host registry and publish nothing, so need no realm; `compaction-basic` + `tool-result-pruner` share one realm because the engine reads the pruner via `ctx.get`; `plan-mode` gets its own because plan state is per-agent by nature. **Default for a new tool: a preset row registering into a host registry, publishing nothing.**

Presenters must be pure functions of their arguments — they run against historical arguments during replay. Thread `exec.signal` into anything that can block; a tool ignoring it delays cancellation with no watchdog to save it.

Also reachable: `exec.deferContext(message)` → a message for the *next* step; `exec.concludeTurn()`; `meta` for replay-time presentation data; `finalizeContent` for content even when the pipeline bypasses post-execute; `timeoutMs`, cooperative and never shown to the model.

34 Composition in full

```
apps/server/src/index.ts (profile: 'web' hardcoded, --profile refused)
  → composeProfile → prepareProfile writes cordis.yml = "[]"
  → boot: new Context() → ctx.plugin(Loader) → mountRootInclude
  → Include[Service.init] → applyEntryPatches → EntryGroup.update (every row CONCURRENTLY)
  → Entry._start → registry.plugin → new Fiber → new AgentLoop(ctx, config)
```

The patch stack, in fixed order (`profile-boot.ts:136-143`): each bundle's `cordis.patch.yml` in `bundles` order → the profile's own → `$DSH_HOME/cordis.patch.yml` → `--patch` overlays.

The patch algebra (`vendor/include/src/index.ts:58-128`): `insert` with no `id` appends top-level; `insert` with an `id` appends into that group's config; `{id, ...overrides}` copies each override key onto the row —

```
for (const [key, value] of Object.entries(overrides)) target[key] = value
```

config is replaced wholesale, not merged. A later layer restating `config`: discards every key the earlier set — which is why the preset restates the pruner's budgets identically rather than relying on the base. Inserted rows are indexed **immediately**, so a later patch in the same list can target a row an earlier patch just added.

Row order is not load order: rows create concurrently, and a fiber activates only when its `inject` dependencies resolve.

`!!js` is a tagged scalar parsed to inert `{__jsExpr: "<source>"} (include:9-23)`. `!!foo` is YAML shorthand for `tag:yaml.org,2002:foo`; a single `!js` would not match the custom Type at all — the repo's rule is mechanical, not stylistic. Evaluation is **lazy and per row** via `internal/config`, **skipping tree-carrier plugins** whose config is entry lists and must stay literal. The evaluator:

```
export const evaluate = new Function('ctx', 'expr', `with (ctx) { ... `)
```

— `vendor/loader/src/config/utils.ts:5-6`

Free identifiers resolve against the row's own context — which is why `ctx.webStartup.host` works in a row that injects `webStartup`, and why an unresolvable identifier throws rather than yielding `undefined`. It also means **!!js is arbitrary code at config-load time**, putting `cordis.patch.yaml` in the same trust class as the code it composes.

Two unrelated isolation mechanisms

Isolate realms (`Cordis/Loader`, `config/isolate.ts`) remap *which symbol a name resolves to* for one entry's subtree. `isolate: {x: true}` mints a `LocalRealm` (suffix `'#+id'`); a string label mints a `GlobalRealm` (`'@'+label`) shared by every entry naming it — but a shared label does **not** pool instances: `provide()` throws on a second registration under the same realm symbol. Labels *join* realms.

The key property: a service published with **no** `isolate`: is visible process-wide because its context inherits the root's symbol table unchanged. **Visibility is the default; a realm is an opt-in narrowing, never a widening.** `mountPreset` enforces it — `leakedServices()` flags any implementation stored at the *root's* symbol, so a row that forgot its realm **fails session creation loudly**.

Scope parentage (`packages/core/scope`) is application-level, built from `ctx.extend()`, a private `WeakMap`, and Cordis's listener-filter hook — it never touches `Context.isolate`. It provides event routing (`scopeTarget`) and registry layering (`ScopedLayers.merge = global + chain`, nearest wins). **One relation, scopeParents, with two consumers.**

Agent presets

`ensureStanding(preset)` (`agent-presets/src/index.ts:747-795`) keys a single-flight map by preset id — and also stamps the composition **file's mtime/size** on every call, evicting and creating a **new generation** if it changed. So the file comment's "mounted once per process" is more precisely **"once per (preset id, on-disk file generation), single-flighted and lazily created on first use."** A live-edited preset forks a generation; sessions already joined keep theirs.

`mountPreset` does `agentCtx.plugin(PresetTree, config)` (`PresetTree` extends `Include`) — plugged directly, not through a Loader entry — then verifies every row activated and no row leaked a service.

Joining is the cheap part. `AgentPresets.mount` is called from the factory's `setup` hook (§14) and does:

```
this.bindings.set(agentKey, bindScopeParent(agentKey, standing.key))
```

One WeakMap entry. No new fiber, no new plugin instance, no module re-import — and because that same link drives both event routing and `ScopedLayers`, the agent immediately sees the preset's tools, sections, and scoped services.

`serviceForAgent` (`mount.ts:259–293`) is a deliberate backdoor for host code needing to read *inside* an agent's realm-private service: it walks the reflect store testing **fiber-tree membership** rather than using the isolate map — bypassing realm invisibility by using a different addressing scheme.

Reading a composition correctly

1. **base** — what exists at all.
2. **web-app patch** — `disabled: true` rows, and `config: replacing wholesale`.
3. **the preset** — re-mounts most of what layer 2 disabled, sometimes with *different* config.
4. Check for `!!js`.
5. Check for **mounted-but-dormant** (`openAt: never`, empty providers).

Skipping step 3 produces confident wrong answers: compaction, subagents, skills, plan mode, and most tools all look disabled after step 2.

The vendored framework

`cordis 4.0.0-rc.7` from `cordiverse/cordis@56b3d4f7` (core + loader); the companions from a **private fork**, `deepseek-harness/cordis@abb0a307`. `vendor/README.md` logs 19 local modifications and asserts the log is exhaustive.

✅ **Audited: the log is accurate — no contradictions, no omissions.** Thirteen items were diffed against the exact pinned baseline. The strongest is item 7, which claims the JSDoc enrichment across seven files is "comment-only; no code changes" — stripping comments leaves **zero functional differences**, every residual being an import rewrite items 4/10/17 already declare or linter whitespace. Item 19 is a genuine bug fix: upstream classifies Node's module loader by `major >= 24`, but the v2 interface landed in **24.12.0**, so 24.0–24.11.1 are mistagged; the vendored copy probes for the actual API.

⚠️ Five items (1, 9, 12, 13, 14) target the private fork, so they were checked for *presence* — all present and matching — but "present here, absent from cordiverse" cannot separate a harness change from a fork change. **This is the one claim in the book unverifiable in this environment.**

35 Limits and fragile areas

1. **The token meter is not a tokenizer.** 4 chars/token, no BPE, nothing provider-specific — wrong in both directions for code, CJK, and long identifiers. Compensated by preferring real usage after one success and by the 80% threshold. Bites hardest when a session's *first* request is already near the window (a large seeded fork), where no usage exists to anchor on.
2. **maxParallelToolCalls is deployment-wide.** No per-agent or per-tool cap; ten parallel file reads and ten parallel network fetches get the same budget, and each subagent inherits it.
3. **Sibling effects tear down concurrently.** Disposers *nested inside one* effect are strict LIFO, but a whole-fiber unload runs all top-level ones with `Promise.all` (`fiber.ts:675–696`). `AgentLoop` registers exactly two; their relative order is undefined.
4. **ctx.emit() does not contain listener failures.** One synchronous throw aborts the rest. The agent dispatcher works around it per-call-site; anything calling `ctx.emit()` directly inherits the hazard.
5. **Provider errors are classified by regex on vendor prose.** If a provider rewords its message, `CONTEXT_WINDOW_EXCEEDED` stops being produced and the symptom is *turns failing where they used to recover*, with nothing in the harness changed. **The most fragile behavior-critical coupling here.**
6. **Cancellation has no watchdog.** A tool ignoring its signal blocks `whenIdle()` → agent disposal → factory teardown (which `Promise.all`s every live agent). One unresponsive tool can stall shutdown indefinitely.

7. **always retry mode is unbounded** — no code gate, no cap; and its fall-through reads as if it returns early when downstream declines, and does not.
8. **Runtime invariants ship switched off** — so the central guarantee is enforced only in CI. Filters also compile once at startup; changing them needs a plugin reload.
9. **agent/turn-stopping has zero live listeners** — the first plugin to use it exercises an untested path.
10. **No working install out of the box (§32)** — the first failure a new user sees is one the engine cannot handle.
11. **No session-format migration path.** `SESSION_FORMAT_VERSION = 0`; mismatches refused in both directions; a structural change would strand every existing session.
12. **Acknowledged in-source:** `TODO(call-config-shape)` flags `LlmCallConfig` as unsettled — and it is the header's `config`, so changing it touches log compatibility; `DSH_TOOLS_MODE` is called a **"TEMPORARY workaround"** standing in for per-session selection; continuable fork invalidates the cache prefix forking exists to preserve.
13. **Smaller edges:** `str_replace_editor` is exclusive even for `view`; a live-edited preset forks a generation; the invariant's `JSON.stringify` comparison is key-order sensitive; a raw `ToolDefinition` gets no argument validation; **spill files are swept by mtime alone**, so a session resumed after 30 days holds dead locators (§26).

36 Design decisions

ADR-1 • The log is the single source of truth. *Alternatives:* an in-memory array persisted periodically (resume needs separate serialization, fork needs deep copy, replay needs a drifting fixture format); log + synchronized cache (two sources that can disagree — and the disagreement surfaces as a session replaying differently than it ran). *Tradeoffs:* 43 events for a one-word turn, and a compression codec had to be written; derivation runs per request, needing an incremental cache with a generation guard. **Bought:** resume/fork/compaction/replay as one operation, and a correctness property that is *checkable*.

ADR-2 • Rewrite by superseding, never mutating. *Alternatives:* delete old events (contradicts ADR-1, user's scrollbar loses content they saw); rewrite in place (breaks provenance — nothing can say what a summary stands for). *Tradeoffs:* an indirection every reader must learn, plus real provenance rules. **Bought:** compaction is a normal append with no special write path, and `replaceGeneration` gives four mechanisms a precise "history changed" signal.

ADR-3 • Registrations are effects. *Alternatives:* explicit `unregister(name)` (callers must remember names; double-unregister becomes a bug class); GC on plugin unload (the framework would need to know every registry — inverting the dependency the plugin system exists to avoid). *Tradeoff:* the concurrent-sibling-teardown nuance. **Bought:** teardown structurally correct by default; `provide()` is an effect too, so *un-providing* becomes a synchronization point.

ADR-4 • The agent plane moves behind presets. *Alternatives:* one global composition (every session identical); a full plugin tree per session (fresh imports and fibers each time, and every process singleton duplicated and colliding). *Tradeoffs:* the hardest thing here to read correctly, plus two similarly-named isolation mechanisms. **Bought:** the marginal cost of a session joining a preset is **one WeakMap entry**, and `mountPreset` mechanically rejects a leaked service.

ADR-5 • Retry is an extension point. *Alternatives:* retry in the loop (one policy shape for every deployment, and compaction-on-overflow would need a separate hook anyway); retry in the adapter (invisible to the log, so no durable budget, and compaction could not participate). *Tradeoffs:* no retries unless something is mounted; the loop cannot bound them. **Bought:** compaction and retry compose on one seam without knowing about each other — and because a retry rebuilds from the log, a listener can *change history* and have the rebuilt request pick it up. That falls out of ADR-1 rather than being designed.

ADR-6 • policy: 'never' is not delegatable. *Alternative:* a prepended listener — but "prepended" is only a position, and another listener can also prepend; correctness would depend on mount order. *Tradeoff:* an asymmetry a reader must notice. **Bought:** a guarantee no configuration can defeat. It pairs with the opposite use of the same insight — §11 uses `prepend: true` precisely *because* a short-circuiting listener could silence it. The codebase knows ordering is unreliable and reasons about it in both directions.

Two principles recur. *Make the correctness property checkable* — ADR-1 produces an assertion, ADR-4 produces `leakedServices()`, ADR-6 produces an undefeatable guarantee. *Prefer loud failure over quiet degradation* — unknown event types refuse a session, a malformed chunk row throws, a leaked service fails mount, an unanswerable approval is denied. The exceptions are narrow and deliberate: spill keeps the original on failure, presenters swallow invalid historical arguments, `emit` listener failures are contained.

Part VI • Rebuild it

37 A milestone path

Ten milestones in dependency order. **Milestones 1–7 need no plugin framework at all** — build the loop against concrete dependencies and extract seams once you know where they belong.

#	Build	Real files	Test it	The trap
1	Event log: <code>append(type, data)</code> , <code>seq = log.length</code> , frozen, deep-snapshotted	<code>core/session/src/index.ts:602–653</code>	10 events → seq <code>0..9</code> , no gaps; mutating your input doesn't change the log	storing a reference instead of a snapshot
2	<code>deriveMessages()</code> — project 3 types, skip the rest	<code>surface.ts:83–114</code> , <code>index.ts:724–745</code>	boundaries absent from the result	adding formatting here instead of at append time
3	The ~100-line loop with a mock adapter	<code>agent-loop/src/agent.ts:219–438</code>	ordered events; the mock received exactly <code>deriveMessages()</code>	keeping <code>this.messages</code> alongside the log
4	Tool calls: <code>append tool/call</code> , <code>execute</code> , <code>append tool/result</code> citing it	<code>tool-calls.ts</code> , <code>tools/index.ts:1028–1053</code>	two steps; second request contains the result	forgetting <code>null</code> means <i>continue</i>
5	The surface: <code>nodes</code> , <code>replaceGeneration</code> , mandatory <code>surfaceOp</code> , <code>replace</code> as splice	<code>surface.ts</code>	5 msgs → replace middle 3 → 3 nodes, all 5 still in the log	letting a replacement cite less than it shadows
6	<code>BlockAssembler</code> keyed by index; <code>block-end</code> authoritative; drop tool calls on <code>max-tokens</code>	<code>llm/src/assembler.ts</code>	interleaved chunks assemble; truncated call dropped	keying by arrival order
7	Cancellation: per-turn <code>AbortController</code> , salvage partials, drain started tools, synthesize results for unstarted	<code>agent.ts:143–232</code> , <code>tool-calls.ts:238–260</code>	log balanced ; a fresh request from the cancelled session is valid	abandoning unstarted calls — this is where "resumable" becomes true or false
8	Inbox: two queues as replayed events; claim all of <code>next-step</code> , one of <code>next-turn</code>	<code>agent/src/inbox.ts</code>	inject on idle → no turn starts	making the queue a plain array
9	Prompt sections with orders; <code>waterfall(listeners, payload, default)</code> ; convert step logic to <code>agent/pre-step</code>	<code>system-prompt:536–611</code> , <code>cordis/events.ts:234–243</code>	a listener not calling <code>next()</code> suppresses the default	assuming listeners always delegate
10	Compaction: threshold, tool-pair-balanced range, summarize, commit as <code>replace</code> , refuse if not smaller	<code>compaction-basic/src/region.ts:154–256</code>	surface shrank, log did not ; a second compaction of a terse session is refused	splitting a tool-call/result pair

Result: ~1,500–2,000 lines that resume, fork, cancel cleanly, and compact. Everything beyond — the four-stage tool pipeline, parallel scheduling, adapters, retry, spill, subagents, persistence, presets — is genuinely optional. **None of it is needed to demonstrate the central property**, which is the strongest evidence that the property is the design rather than a consequence of the size.

Appendices

A • Glossary

adapter — knows how to talk to one or more providers; only `stream()` is required. Retry policy captured at registration. §23 **adapterDefaults** — record of which config fields the *adapter* supplied, so the next request strips and re-resolves them. §10, §23 **agent plane** — what *one agent* contributes: tools, prompt sections, delegation backends. Mounted per preset. §34 **barrier** — an exclusive tool call that runs alone. §17 **canonical result** — already validated and rendered through its tool's output contract; re-normalizing is a no-op. §16 **compaction transaction** — `compaction/start ... compaction/end`; the start event is the lock, so it survives a crash. §27 **Cordis** — the vendored plugin/DI framework. Context · Service · Fiber · Effect. §2, §34 **context sections** — prompt contributions that become a runtime-context *message* rather than system-prompt text, because they change mid-session. Three producers exist. §19, §21 **durable-first** — commit to the log *before* mutating memory, so observers see the pre-change state and it survives a crash. §12 **effect** — `ctx.effect(fn, label)`; every registration in the codebase is one. §2, §36 **fiber** — one plugin instance; `PENDING` until every `inject` resolves; unloads when one disappears. §2 **host plane** — mounted once, visible everywhere: registries, sandbox/approval, persistence, model routes. §34 **inbox** — two queues (`next-turn`, `next-step`) reconstructed by replaying `agent/inbox/spliced`. §12 **invariant companion** — a package-shipped runtime check of a relationship it owns; ~30 exist, the registry runs nowhere. §11 **isolate realm** — remaps which symbol a service name resolves to for one subtree. Visibility is the default; a realm narrows. §34 **maintenance** — a phase where the agent holds still but reports `idle`. §13 **markAgentLoopRequest** — process-local `WeakSet` tag marking a request loop-built vs one-shot; never serialized. §10 **phase** — `idle · maintenance · running`, collapsing to two external statuses. §13 **prepared call** — frozen, single-use, config-checked, pinned to the adapter generation. §23 **preset** — a named agent-plane composition, mounted once per (id, file generation), joined by one `WeakMap` entry. §34 **profile** — not a file: an empty root config plus an ordered patch stack. §2, §34 **projection** — a registered, versioned, pure fold over the log. §8 **PromptAssembly** — resolved but uninterpolated sections/contexts/tools/variables. §19 **prompt variable** — `(context) => string | undefined`, evaluated once per assembly, substituted at render. The engine registers `provider`, `model`, `cwd`. §20 **PTC / run_code** — a tool-presentation mode where the model writes a program that calls tools. Behind `DSH_TOOLS_MODE`; default `native`. §15, §16 **request header** — the non-message half of a request; logged only on change, four reasons. §10 **replaceGeneration** — bumped once per committed replace; the system's "history was rewritten" signal, read by four mechanisms. §6 **seq** — an event's index in the log. Assigned in one place, verified in four. §5 **SessionPreparation** — `Disposable` wrapper around one *unpublished* session. §29 **snapshot message** — the synthetic runtime-context `user/message`, injected only when its text changed. §21 **spill** — writing an oversized plain-text tool result out of line at execution time. §26 **step** — one model request within a turn. §9 **surface** — the ordered list of log positions the model currently sees. §6 **surfaceOp** — mandatory on surface-eligible events: `'append'` or `{op: 'replace', start, end}`. §6 **subagent** — a genuine nested `Agent` from the same factory; `spawn` empty, `fork` seeded. §28 **token meter** — real provider usage preferred, else 4-chars-per-token heuristic. Not a tokenizer. §26 **turn** — one unit of conversation. **turn ending** — `completed · max-tokens · blocked · aborted · error · interrupted`. §9 **waterfall** — nested continuations; a listener returning without calling `next()` vetoes everything after it *including the default*. §22 **wake latch** — a deferred wake replayed at convergence, never during disposal. §13

B • Data-structure reference

SessionEvent<T> `types.ts:391-398` — `seq · time · type · data · ignorable? · surfaceOp? · sourceEventSeqs? · SessionHeader` :56-94 — `version · id · createdAt · cwd? · parentSession? · seedLength? · origin? · delegationDepth? · agentPreset?` — kept **out** of the log **SurfaceOp** :359-361 —

'append' | {op:'replace', start, end} (start/end name **surface nodes by seq**) **SessionSurface** surface.ts:137-142 — nodes: readonly number[] · replaceGeneration: number **EpochHeader** :179-188 — config: LlmCallConfig · adapterDefaults? · system? · tools? **ChunkRow** chunk-rows.ts:66-69 — {type: text|reasoning|tool-call -chunks, seq0, time0, data{turn,step,index,dt[],texts|args[],id?,name?}} — **not an event**

Message message.ts:131-140 — id · role · content: ContentBlock[] · source **MessageSource** :102-107 — user · plugin{plugin} · model{provider,model,replayState?} · tool{callId} **ContentBlock** types.ts:54-110 — text · reasoning · image · tool-call{id,name,arguments} (raw JSON string) · tool-result{toolCallId,content,isError?} **StreamChunk** :364-376 — block-start · text-delta · reasoning-delta · tool-call-delta · block-end · usage · finish **FinishReason** :116-125 — stop · tool-calls · max-tokens · aborted{failure} · error{failure} **GenerateOptions** :393-429 — provider · model · reasoningEffort? · messages · system? · tools? · temperature? · maxTokens? · stop? · signal? · sessionId? · purpose? **LlmCallConfig** call-config.ts:23-30 — provider · model · reasoningEffort? · temperature? · maxTokens? · stop? · ⚠ carries TODO(call-config-shape) **LlmFailure** :40-51 — message · code · status? · providerRetryAfterMs? · requestId? **PreparedLlmCall** llm/index.ts:157-177 — config · retryPolicy · context?{contextWindow} · inputModalities? · adapterDefaults · stream() (single-use)

ToolDefinition tools/index.ts:214-280 — ToolSchema + output · execute · finalizeContent? · timeoutMs? · isConcurrencySafe? · presentCall? · presentResult? **ToolExecutionInput** :307-331 — callId · rootCallId? · name · arguments · agent? · parent? · signal **ToolRunContext** :397-414 — adds deferContext(msg) · concludeTurn() **ToolExecutionResult** :549-573 — success {isError:false, value, content, meta?, additionalContexts?, concludesTurn?} / failure {isError:true, error, content, meta?, additionalContexts?}. **value never persisted**; concludesTurn typed never on failures **PreToolDecision** :576-584 — allow | deny{reason} | ask{reason?} **ScheduledToolPreparation** :428-437 — dispatch{exec} | post-result{result} | final-result{result} **Constants** — TOOL_ABORTED='ABORTED' · TOOL_ABORTED_BEFORE_DISPATCH='ABORTED_BEFORE_DISPATCH' · TOOL_RUNTIME_SCHEDULER · DEFAULT_MAX_PARALLEL_TOOL_CALLS=10 · SESSION_FORMAT_VERSION=0 · MIN_RUN=3

PromptAssembly system-prompt:114-119 — sections · contexts · tools · variables (uninterpolated) **Phase** agent.ts:39-47 · **InboxTarget** 'next-turn'|'next-step' · **PreStepDecision** reject | enter{messages, startsRequestSeries?} · **RequestErrorAction** {kind:'retry'} | undefined **ProjectionDefinition** session-projection:42-86 — key · stateSchema · init · apply · wire? · stateVersion

C · Configuration reference

A value reaches a session through up to three layers, last write winning per row id, with config **replaced wholesale**: base → web-app patch → standard preset. **For any per-agent tool, the preset's value is the live one.** Two rows where they differ: tool-web fetch: false → true; tool-subagent-fork one-shot → **continuable**.

Setting	Live default	Where
maxParallelToolCalls	10 — integer ≥1; deployment-wide	constants.ts:6
agents	[]	base:486-489
default route	deepseek-official / deepseek-v4-flash	base:73-79
llm-deepseek	disabled: true	web-app:41-42
llm-pi-ai providers	{} — zero routes	config.ts:340-342
defaultContextWindow / DEFAULT_MAX_TOKENS (DeepSeek)	1_000_000 / 256_000	adapter.ts:140,142
reasoningEffort (DeepSeek)	high; off only when thinking: disabled	adapter.ts:187-193
retry maxRetries / delays / jitter	5 / 500 – 10_000 ms / 0.1	retry-policy.ts:14-24
retryableCodes	EMPTY_RESPONSE, RATE_LIMIT, SERVER, TIMEOUT, TRANSPORT	same
compaction auto / thresholdRatio / retainRatio	true / 0.8 / 0.16	config.ts:95,20,23
maxOverflowRetries	1	config.ts:93
pruner thresholdChars / headChars / tailChars	8192 / 4096 / 1024	base:404-409, preset:150-155
maxInlineBytes (spill)	50000 — host-plane, untouched by web-app	base:393-396
cleanupPeriodDays (spill)	30; 0 disables	spill-local:68
token estimate	CHARS_PER_TOKEN=4, BLOCK_OVERHEAD=4, ROLE_OVERHEAD=4	estimate.ts:13-19
sandbox mode / workspaceRoot	workspace-write / process.cwd()	base:217-218
approval policy	ask; never iff DSH_PERMISSION_MODE === 'danger-full-access'	base:233
presets	read-only / workspace-write / danger-full-access	base:238-247
persistence root / compression / packChunks	dshHomePath('sessions') / zstd / true	base:110-113
session-query-sqlite	:memory:, openAt: never	base:129-133
projection cache	writeEveryEvents: 200, writeIntervalMs: 5000	base:162-166
subagent maxDepth	3, or 'provider-managed'	tool-subagent:129
agent-instructions.maxBytes	65536	base:274-277
tool-ralph.maxRounds	64	base:422-427
repeat-tool-reminder	thresholds: [3,5,8], argumentsPreviewChars: 500	base:434-438
tool-web.searchTimeoutMs	60000	preset:231-235
agent-presets.default	standard	web-app:451-456
webserver	127.0.0.1:3080, gzip lvl 1 above 1024 B	web-app:121-129

Environment: DSH_PERMISSION_MODE (sandbox mode **and** derived approval policy) · DSH_TOOLS_MODE (native | ptc | both ; unset = native ; **documented temporary**) · DSH_TELEMETRY_MODE (FEEDBACK_ONLY default) · DSH_TELEMETRY_DISABLED (**any** non-empty value, including '0' / 'false') · DSH_TELEMETRY_OTLP_URL · DEEPSEEK_API_KEY · DSH_HOME .

Platform-conditional (lazily evaluated per row, entry.ts:100-108): tool-bash / bash-sandbox disabled on win32; tool-pwsh / pwsh-sandbox disabled off-win32.

Mounted but inert: session-query-sqlite (never opened) · llm-pi-ai (zero routes) · hmr , skill-badge (disabled in base) · ui-schedule (disabled in web-app) · dsh-invariants (mounted **nowhere**).

D • Mechanism index

Mechanism	§	Key files
Append-only log	5	core/session/src/index.ts:602–653
Surface	6	core/session/src/surface.ts
Derivation	7	surface.ts:83–114, index.ts:724–745
Projections	8	session/session-projection/src/index.ts
Turn / step loops	9	agent-loop/src/agent.ts:255–438
Request construction	10	agent-loop/src/agent.ts:444–544
Reconstruction invariant	11	agent-loop/src/invariant.ts
Inbox	12	core/agent/src/inbox.ts
Phases / cancellation	13	agent-loop/src/agent.ts:39–232
Agent lifecycle	14	agent-loop/src/index.ts:522–773
Tool registry	15	core/tools/src/index.ts:1028–1053
Execution pipeline	16	core/tools/src/index.ts:1450–1667
Tool scheduling	17	agent-loop/src/tool-calls.ts
Approval / escalation	18	interaction/user-approval/src/index.ts:222–309, sandbox/src/escalation.ts:157–189
Prompt assembly	19	core/system-prompt/src/index.ts:536–611
Interpolation	20	core/system-prompt/src/index.ts:309–346
Runtime context	21	agent-loop/src/runtime-context.ts
Extension points	22	core/agent/src/dispatch.ts, vendor/cordis/src/events.ts:194–243
Adapters / prepareCall	23	llm/llm/src/index.ts:890–935
Block assembly	24	llm/llm/src/assembler.ts
Failures / retry	25	llm/src/adapter-failure.ts, llm-retry/src/index.ts:194–258
Metering / spill	26	llm/token-meter/src/, spill/spill-policy/src/index.ts:110–232
Pruning / compaction	27	compaction-basic/src/{index,region,summarizer}.ts
Subagents	28	subagent-in-process-driver/src/index.ts:103–234
Persistence	29	session-persistence/src/coordinator.ts, ...-jsonl/src/index.ts
Crash repair / packing	30	core/session/src/repair.ts:28–134, chunk-rows.ts
Boot / patches / realms / presets	34	apps/server/src/, vendor/include/src/index.ts:58–128, vendor/loader/src/config/isolate.ts, preset/agent-presets/src/{index,mount}.ts

Extension points: agent/pre-step (waterfall, **12** live) • agent/request (1) • agent/request-error (2) • agent/turn-stopping (serial, **0**) • agent/status (3) • agent/created / disposed (5/4) • agent/inbox/* • agent/error (3) • agent/session-start (2) • tools/pre-execute (**o producing ask**) • tools/execute (timeout policy) • tools/post-execute (spill) • tools/result • llm/stream (invariant — not mounted) • system-prompt/assemble • approval/request • session/event.

Evidence

Written entirely from reading the implementation, then checked by building and running the suite.

packages/core/agent-loop	342 / 342 pass, 18 files
core + session + llm + compaction + spill	3,373 / 3,383 pass, 165 files
The 10 failures	one file, all EPERM ... symlink — Windows blocks symlink <i>creation</i> ; environmental
test:snapshot	67 fail — POSIX fixtures on Windows: expected "name": "bash", received "name": "pwsh", exactly as the documented platform gate requires
Vendored changelog	all 19 items audited against cordiverse/cordis@56b3d4f7 — no contradictions

Nothing was rewritten to fit a test result. **Three claims in the first draft were wrong**, all failing the same way — a plausible pattern mistaken for evidence: a suggestive name (reasoning-chunks , actually a lossless storage codec), a grep matching one call shape (register(defineTool , missing four tools and two raw registrations), and a search scoped to src/ while the producer sat in YAML (complete: true). None would have been caught by review; each needed someone to open the file.

The one claim unverifiable in this environment: the attribution of five vendored modifications to local rather than fork authorship, because deepseek-harness/cordis is private.

Appendix E • Why each mechanism is more than its minimal version

Every mechanism here has a version you could write in a dozen lines. This appendix is the difference: for each, the minimal form, then **what the real implementation adds and the specific failure each addition prevents**. It is the densest statement of the engine's accumulated hard-won knowledge.

The log (§5)

```
append(type, data) { const e = Object.freeze({type, seq: this.log.length, time: Date.now(), data});
this.log.push(e); return e }
```

Addition	Failure it prevents
Lossless-JSON snapshot	A payload that cannot round-trip becomes corrupt storage discovered much later
Freezing	Callers share references; a caller could mutate logged history
Reentrancy refusal	A listener appending during dispatch breaks <code>seq = log.length</code>
Pre-commit surface validation	A bad marker must fail with nothing mutated, not leave a half-valid surface
Contained observers	One bad listener must not un-commit history
<code>ignorable</code> + generated known-type gate	Vocabulary must grow without breaking old readers, and fail loudly when it cannot
Cached <code>events</code> snapshot	Readers ask constantly; re-copying the array each time is wasteful

The surface (§6)

```
apply(seq, op) { if (op === 'append') this.nodes.push(seq)
  else { const s = this.nodes.indexOf(op.start), e = this.nodes.indexOf(op.end)
    this.nodes.splice(s, e - s + 1, seq); this.replaceGeneration++ } }
```

Addition	Failure it prevents
Closed eligibility + mandatory intent	An event silently defaulting to "append" corrupts history undetectably
Provenance coverage	Without it a replacement's meaning depends on range bounds later rewrites can shift
Strictly-earlier source check	Citing a later event makes the log unreplayable in order
Narrow <code>tool/result</code> rewrite	A general rewrite could change <i>whether a tool failed</i> ; only its text should shrink
Append-origin guards	Human and model transcripts genuinely diverge after a compaction
Incremental fold + pure <code>foldSurface</code>	The hot path reads this constantly; offline readers must rebuild from nothing

Derivation (§7)

```
deriveMessages() { return this.surface.nodes.map(s =>
  deriveEventMessage(this.log[s])).filter(Boolean) }
```

Addition	Failure it prevents
Append-only cache + cursor	Every step re-derives; re-walking the whole surface each time is wasted work
Generation-guarded invalidation	A replace edits the <i>middle</i> of the node list, so an append-only cache is invalid
Fresh array, shared frozen messages	Callers must not see each other's mutations; deep-cloning frozen data is pointless

What is *absent* is the lesson: no reformatting, no framing, no merging of adjacent messages. Each would have baked a decision into replay.

Projections (§8)

```
function project(events, init, apply) { return events.reduce(apply, init) }
```

Addition	Failure it prevents
A registry keyed by name	Consumers each re-walking the log
<code>stateVersion</code>	A changed fold forward-applying old caches into garbage
RefCounted registration	The same package mounts under several presets
<code>Object.is</code> double debounce	Most events are irrelevant to most folds; notifying on every append floods clients
Checkpoint / <code>restoreFloor</code> / hydrate	A cold start re-folding a 10,000-event log
Separate <code>state</code> and <code>wire</code>	Host folds often hold more than a browser should see

The turn and step loops (§9)

Addition	Why it exists
<code>null</code> vs an ending as the step's return	Distinguishes "keep going" from "done" without a second flag
Sticky <code>max-tokens</code>	A later clean step must not mask a truncated answer
<code>turn-stopping</code> re-test	Gives plugins a final chance to continue, without letting listener order decide
Fresh abort controller per turn	Cancellation should scope to a turn, not kill the agent
Structured error endings	Replay and UI need a code, never a bare string
Abort check before each boundary append	Prevents an unbalanced <code>step/start</code> with no <code>step/end</code>
Interrupted-message salvage	Cancelling mid-answer should not discard what was already said
Retry delegated to a waterfall	Retry policy is deployment-specific; the loop should not own it

Request construction (§10)

Addition	Why it exists
<code>adapterDefaults</code> + <code>requestProposal</code>	Otherwise one model's defaults silently follow you to another model
Three-condition effort restoration	A persisted effort level is only meaningful on the model that had it
<code>agent/request</code> waterfall	Model switching must not require touching the loop
<code>NO_ADAPTER</code> -only fallback	Middleware may serve unregistered routes; every other failure is real
Change-only header logging	A copy of the tool schemas on every step would dwarf the conversation
<code>series</code> reason	A byte-identical header can still begin a new series after a rewrite
<code>deepFreeze</code> + <code>markAgentLoopRequest</code>	Lets an observer tell "derived from the log" from "hand-built" — and prevents rewriting the former

The invariant (§11)

Addition	Why it exists
A <code>WeakSet</code> tag scoping the check	One-shot calls are legitimately different and must not fail
<code>prepend: true</code>	A short-circuiting listener could otherwise silence it
Frozen checks on request <i>and</i> messages	Catches mutation-after-build, which equality alone would miss
The header half	Messages alone are not the whole request
Package-attributed errors	A violation should name its owner without the checker importing product code
A registry with filters	On the hot path, this must be switchable per deployment

The inbox (§12)

Addition	Failure it prevents
Durable <code>agent/inbox/spliced</code> events	A crashed agent losing pending input
Append-before-mutate ordering	Observers unable to see the pre-splice state
Duplicate-id validation	The same message claimed twice, appearing twice in history
<code>claimed</code> distinct from <code>discarded</code>	"Consumed" and "thrown away" conflated for anything watching
Replay from <code>seedLength</code>	A fork double-counting inherited history
Abort-time retargeting	A message racing a cancellation swallowed by a dying turn
Throw on malformed replay	Running on a queue that cannot be reconstructed exactly

Phases and cancellation (§13)

Addition	Why it exists
A third phase (<code>maintenance</code>)	Some work needs the agent still without looking busy
Status collapsed before comparison	A maintenance cycle should not flicker listeners
Per-phase, per-turn abort controller	Cancellation should scope to a turn
The wake latch	A wake arriving during the aborting window would be lost
<code>disposed</code> excluded from latching	Otherwise teardown waits on a turn it just cancelled
<code>hasPending</code> guard on replay	A latch whose message was cleared starting an empty turn
Clearing the latch on a new controller	A latch against a dead controller is meaningless
<code>whenIdle</code> 's re-check loop	A replayed wake installs new activity before old awaiters run

Agent lifecycle (§14)

Addition	Failure it prevents
Undo registered before resources exist	An unload mid-construction leaking a running agent
Memoized <code>dispose</code>	Racing owners starting parallel teardowns of the same agent
Three fused abort sources	A caller-cancelled create still running because only the factory was checked
<code>assertLive()</code> after each announcement	A listener starts teardown and the agent finishes wiring itself anyway
Enter both registries before announcing	An observer finding a half-entered agent
<code>releaseAbandoned</code> on a raced load	A cancelled disk load completing and leaking its preparation
Re-check ownership after the load	The factory shut down while the disk spun
Factory awaits startup tasks	Teardown completing while an agent is still being created
<code>exists</code> check before create-fallback	A corrupt session silently replaced by an empty one

The tool registry and pipeline (§15–16)

Addition	Why it exists
<code>ScopedLayers</code> instead of a Map	One registry must serve many agents with different tool sets
Mandatory <code>output</code> schema + <code>render</code>	Replay and UI need structured data, not prose
Strict <code>execute</code> , lenient presenters	Bad args must not run; old args must still render
Snapshotted <code>finalizeContent</code>	A hot reload mid-call applying the wrong tool's finalizer
Four separable stages	The scheduler must overlap dispatch while keeping policy and commits ordered
Collapse check before any listener	Never ask a human to approve a call that cannot run
Two cancellation checks in <code>prepare</code>	Cancellation can land before <i>or during</i> the policy await
<code>ABORTED</code> vs <code>ABORTED_BEFORE_DISPATCH</code>	Whether the body ran changes whether a retry is safe
Signal re-fusing	A wrapper must not be able to detach caller cancellation
Canonical-result marking	A result crossing several wrappers being re-validated each time
<code>needsPost</code> by producing stage	Denials deserve post-processing; impossible calls do not

Tool scheduling (§17)

Addition	Why it exists
Per-call classification	A shell command and a file read have different safety properties
Barrier grouping	An exclusive call must see the effects of everything before it
Bounded pool	Unbounded parallelism exhausts file handles and provider limits
Contiguous ordered commit	Completion order is not model order; history must be model order
Live reclassification	The registry can change mid-group
Per-group cap read	A settings change must not disturb a batch in flight
Synthetic results on abort	An unbalanced call/result pair breaks the next request
Drain-don't-abandon	Started work must settle before the turn unwinds
No synthetic results on scheduler failure	Inventing results would hide a bug

Approval (§18)

Addition	Why it exists
Both halves logged with a shared id	"Did someone approve this?" is asked later, about a resumed session
Refusal outside an open turn	An audit event belonging to no turn is unreadable
<code>never</code> checked inline, not as a listener	No registration order may leak a grant
Deterministic degradation to <code>deny</code>	An unanswerable question must never become an allow
Strictly-wider check before prompting	Prompting for a no-op trains people to click yes
Escalation as tool arguments	The request and its justification become durable log content
Conditional schema advertisement	Don't tell the model about a capability that cannot apply
Policy stated in the prompt	Enforcement and instruction must agree, or the model wastes turns

Prompt assembly and interpolation (§19–20)

Addition	Why it exists
Scope-chain shadowing	Two agents in one process need different prompts
Centrally-assigned orders, ≥10 apart	Contributors cannot negotiate; gaps allow later insertion
Code-unit tie-breaking	Locale-aware sorting would reorder prompts per machine and break caching
Sections vs contexts	Volatile content must not sit in the cache-sensitive prefix
Accumulating tool providers	Every tool package contributes; none should shadow another
Empty-render dropping	Conditional guidance without a conditional mechanism
Cursor advancing past each value	A <code>{{</code> inside a substituted value becoming an instruction — template injection
Lone <code>{{</code> tolerated	Prompt text legitimately discusses braces
Unknown vs valueless as separate errors	A typo and an absent provider need different fixes
<code>Object.hasOwn</code>	<code>{{constructor}}</code> resolving to a stringified function

Runtime context (§21)

Addition	Why it exists
Reconstruct from the log on construction	A resumed agent re-announcing what the model already knows
Three states, not two	"Never said" and "said but compacted away" need different handling
Watch replacement events	Compaction silently removing the snapshot the model relies on
The <code>CLEARED</code> sentinel	Going quiet leaves the model holding a stale snapshot
<code>sections</code> on the message source	Attribution for UIs, and a diff showing <i>why</i> it changed
Injection as the waterfall's innermost default	Listeners inherit it by delegating and suppress it by not

Extension points (§22)

Addition	Why it exists
Three distinct modes	Observation, participation, and transformation need different guarantees
Scope-filtered delivery	One process runs many agents; listeners must not cross-talk
Agent fused into the payload	A caller dispatching about another agent
Per-listener containment in <code>emit</code>	Raw <code>ctx.emit</code> lets one throw abort the rest
<code>prepend</code>	Some checks must run before anything that could short-circuit
Dispatcher built once per agent	The hot path dispatches several times per step

Model I/O (§23–25)

Addition	Why it exists
Resolve before dispatch	The header must be logged from the same resolution that is sent
Frozen, single-use prepared call	A retry silently reusing a stale resolution
Config equality at dispatch	Prepare-with-one, dispatch-with-another would make the log lie
Closure captured at prepare	A settings change mid-request mixing generations
Validated model metadata	A bad <code>contextWindow</code> silently breaking compaction's threshold
Registration-time retry capture	Policy is a property of the route, not of a request
Index-keyed partials	Blocks interleave; arrival order is not identity
<code>block-end</code> authoritative	The provider's structured close beats reconstructed deltas
Dropping tool calls on <code>max-tokens</code>	A truncated call's arguments are incomplete JSON
Throwing on an open unknown block	Guessing at an unfinished plugin block fabricates content
Normalization at the adapter boundary	Downstream seeing a raw thrown value
Property-descriptor reads	An error object's getter throwing <i>during error handling</i>
<code>UNKNOWN</code> for foreign codes	Another library's taxonomy treated as this one's
Durable retry counting	A budget in memory resets on restart
Append-before-wait	A crash during backoff leaving no evidence

Context management (§26–27)

Addition	Why it exists
Prefer real usage over the estimate	The provider's count is authoritative once one call succeeds
The $\geq$ -anchor guard	A smaller reported usage under-reporting and compacting too late
Spill as seam + backend + policy	Storage location, retention, and the decision are independent concerns
Retrieval hint naming existing tools	A bespoke retrieval tool the model must learn
Plain-text-only	Spilling an image reference breaks it
The <code>read</code> exclusion	<code>read</code> $\rightarrow$ <code>spill</code> $\rightarrow$ <code>read</code> loops
Best-effort degradation	A failed save losing the tool's output
Prune before summarizing	The cheap deterministic fix often suffices
Tool-pair-balanced boundary	Splitting a call from its result invalidates the next request
Durable <code>compaction/start</code> lock	A mutex does not survive a crash or appear in a log
Prefix reuse for summarization	Keeps the provider's KV cache warm
Refusing a non-shrinking compaction	A terse session compacting repeatedly for nothing
<code>replaceGeneration</code> check before claiming retry	Claiming a retry when nothing changed
<code>compaction/end</code> even on failure	A wedged lock freezing the session

Delegation and durability (§28–30)

Addition	Why it exists
Named providers	<code>spawn</code> and <code>fork</code> differ only in seeding; more backends can register
Depth cap floored by persisted depth	A resumed parent forgetting how deep it is
Preset composition via <code>composeFrom</code>	A child with no tools is useless
Stop-reason mapping	"It stopped" is not enough; the parent needs to know why
Host-plane registry	Cross-session queries and unique provider names
Backend / coordinator split	Every backend reimplementing crash handling, differently
<code>link()</code> over <code>rename()</code>	A second process silently destroying the first's file
<code>fsync</code> + truncate rollback	A half-written line breaking every future load
Reservation with mutation check	Serving a stale prepared session to the next caller
Rejecting an already-live id	Two live sessions diverging under one identity
Live-prefix adoption	Crash-repairing a turn that is still running
Fork rejecting an open turn	A child whose first request is invalid
Per-call result synthesis (repair)	A dangling tool call making the next request invalid
Two distinct repair codes	Retrying a call that may have run is dangerous
Reusing the last real time	Inventing a future timestamp corrupts ordering
Exact-shape whitelisting (packing)	An unrecognized variant losing data rather than compression
Safe-integer gap check	A rounded gap decoding to a different timestamp
Throwing on a malformed row	Silently dropping a whole run
Slash-less row tags	A reader mistaking a storage row for an event